

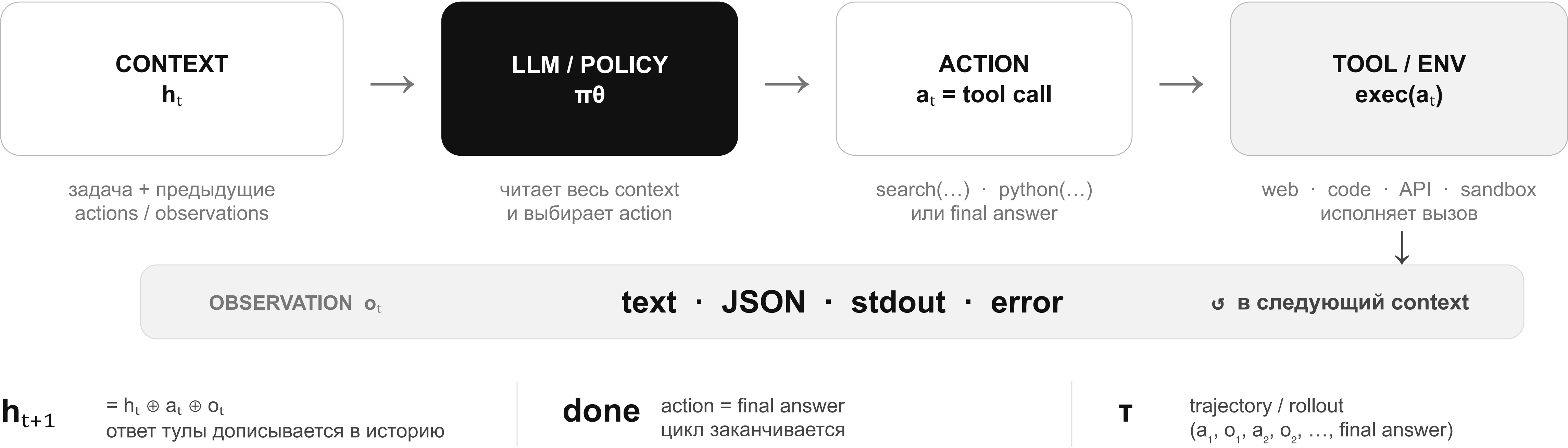
~~Агентный RL~~

Распределённая борьба с пожарами

Несколько обозначений

Агент — это LLM в цикле с инструментами

Action — вызов инструмента. Observation — его ответ в контексте следующей итерации



ИТОГО

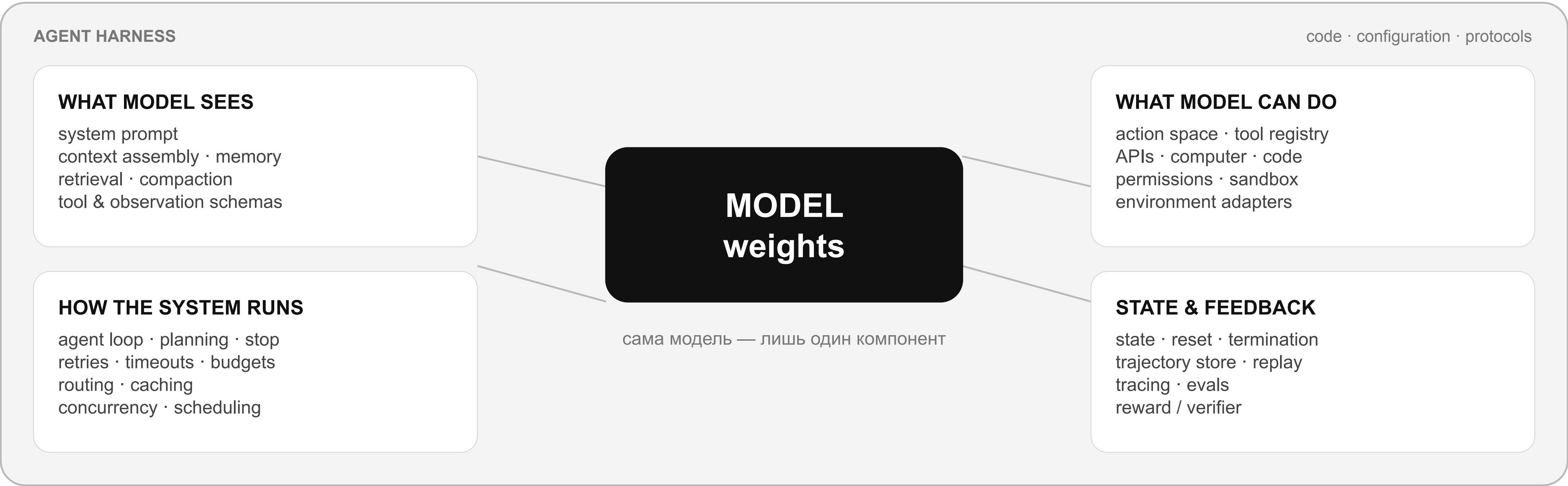
Ответ тулы становится контекстом следующего шага

agent = LLM + tools + context loop · после rollout: reward $R(\tau)$

repeat until done

Harness — программная система вокруг модели

Он определяет, что модель видит, что может делать и как исполняется агент



ВЫВОД

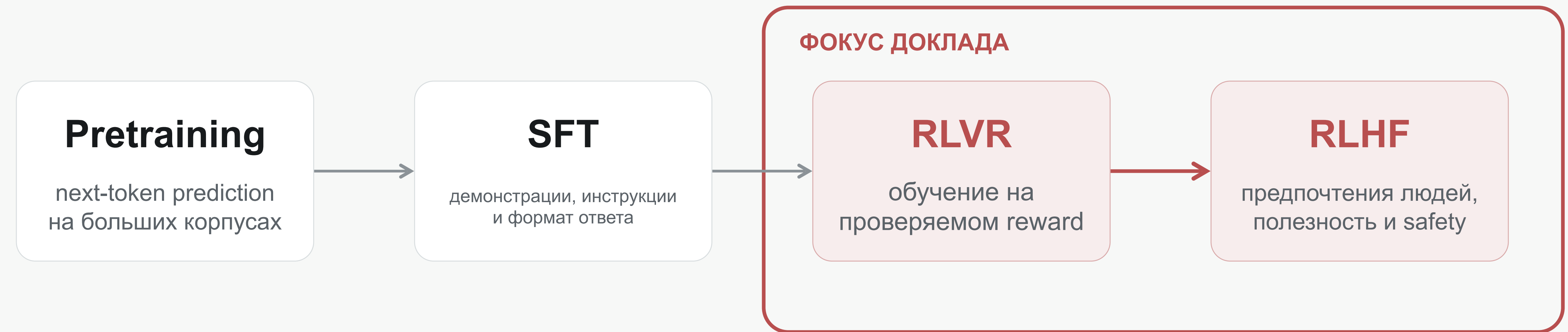
Поведение агента = model × harness × environment

Меняешь harness → меняются actions, observations, reliability и cost

agent loop — часть harness

Условный цикл обучения модели

Удобная карта процесса — не обязательный порядок для каждой модели.



На практике этапы повторяются, смешиваются и могут идти в другом порядке. Здесь это навигация по докладу: дальше — RLVR и RLHF.

Подбираем harness

Наш текущий harness

Запрос + история + файлы + user meta



System prompt + world state



Model, Alice Ai / Kimi / Qwen ...



Tool dispatcher: поиск, страницы, файлы,

Python, медиа, организации и товары



Context manager → reducer → Futuris renderer



Структурированный ответ со сносками и карточками

Web:

- web_snippet → широкий поиск кандидатов
- web_search → сфокусированный поиск
- fetch_url → углубление в конкретный документ.

Вертикали: image_search, video_search, geo_search, ecom_search.

Файлы и вычисления: python_interpreter, file_rag, ocr, fetch_img, image_namer.

Воспроизводим BrowseComp

Сэтап	BrowseComp	Комментарий
kimі-k2.6, изначальный	4	Первое что мы собрали
kimі-k2.6, 10->50 итераций	10	Покрутили итерации
kimі-k2.6, snippet+fetch_url	20	Добавили 2 тулы как в openсорсе
kimі-k2.6, суммаризация в fetch url на той же kimі-k2.6	23	Добавили суммаризацию в fetch_url
kimі-k2.6, суммаризация на gpt5	32	Суммаризацию сделали gpt-5
kimі-k2.6, суммаризация на qwen	38	Поймали оверфит на qwen
kimі-k2.6, yandex->google	40	Сниппетный поиск Яндекса на гугловый
kimі-k2.6, контекст в максимум	50	Не ограничиваем итерации и контекст
Qwen, контекст в максимум	40	Берем qwen

BrowseComp

Выводы

- Многие opensource оверифитнуты в конкретные версии harness и в конкретные тулы
- В итоге сделали BcEnv(snippet + fetch_url + python) и prod env(все тулы)
- Kimi/qwen на бенчах в BcEnv были +10 пунктов на бенчах относительно prod env
- После рл наша модель в prod env сравнима с BcEnv
- Мощность harness зачастую раскрывается только после RL

AGENTIC RL


ROLLOUT
OPTIMIZATION


ACTOR
OPTIMIZATION


ALGORITHMS
GALORE


ASYNCHRONY &
DISTRIBUTION


REWARD
EVALUATION


COMPLEX DATA
FOR TRAINING


AGENTIC
HARNESS


PRETRAIN
STARTING POINT

GIANTS

OF RL RESEARCH & ENGINEERING

MDP

VALUE
ESTIMATION

POLICY
GRADIENTS

EXPLORATION

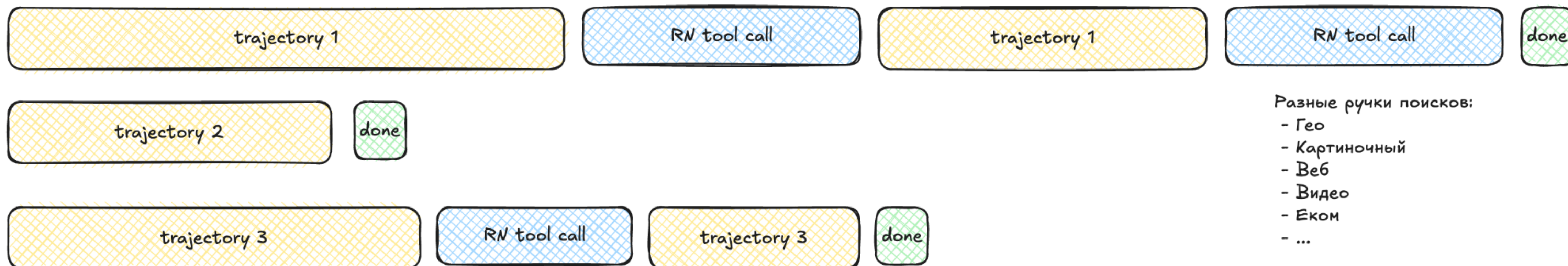
OFF-POLICY
LEARNING

CREDIT
ASSIGNMENT

Asynchronous RL

Агентный rl фреймворк

Синхронный луп

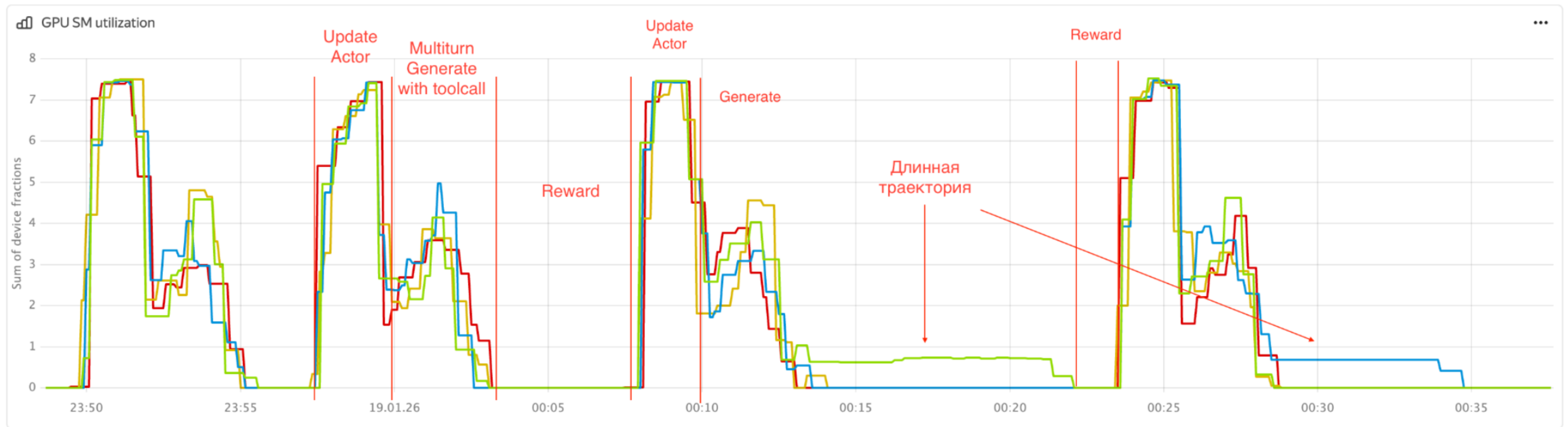


Разные ручки поисков:

- Гео
- Картиночный
- Веб
- Видео
- Еком
- ...

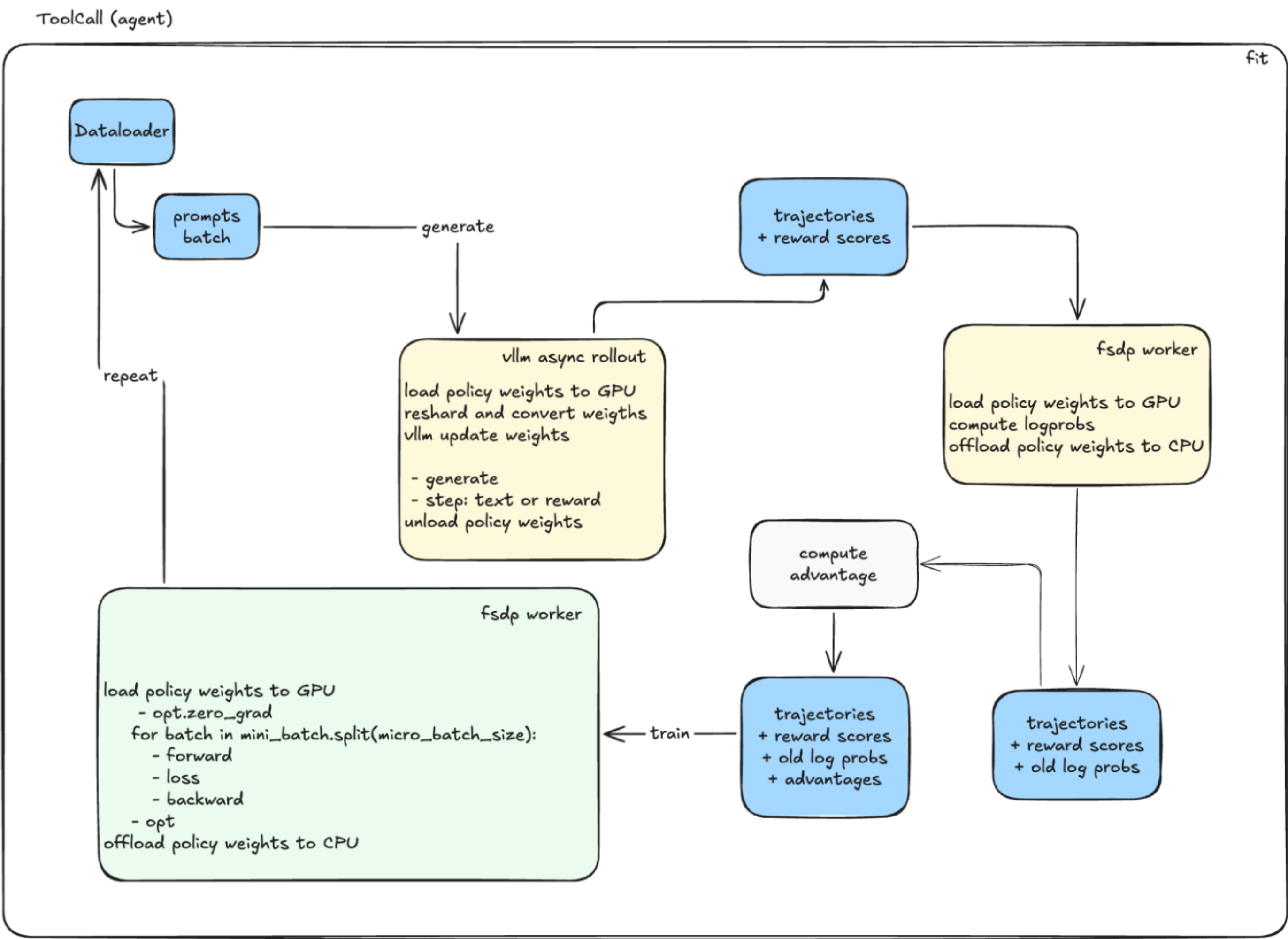
Агентный rl фреймворк

Проблема



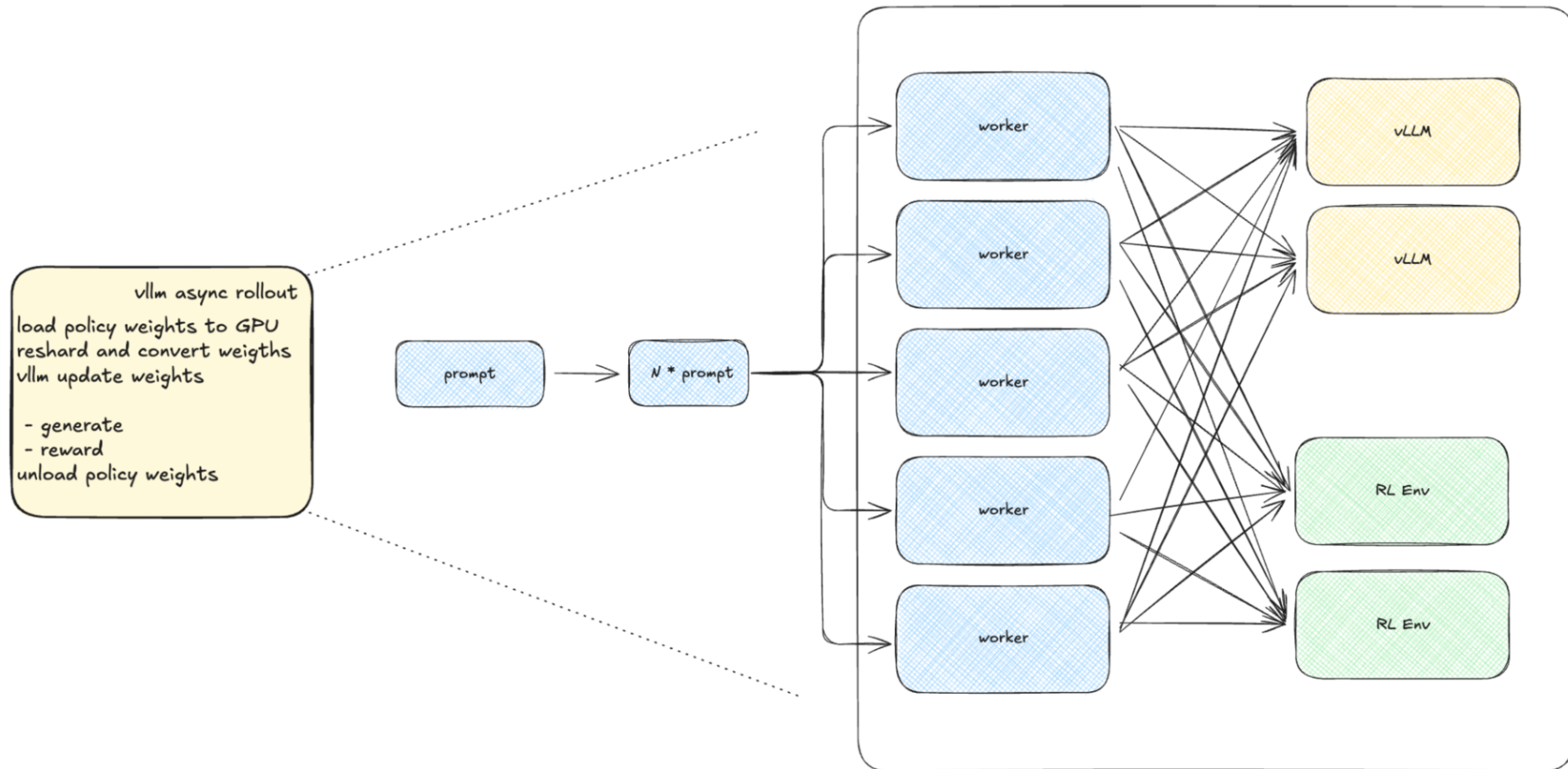
Агентный rl фреймворк

Async Policy



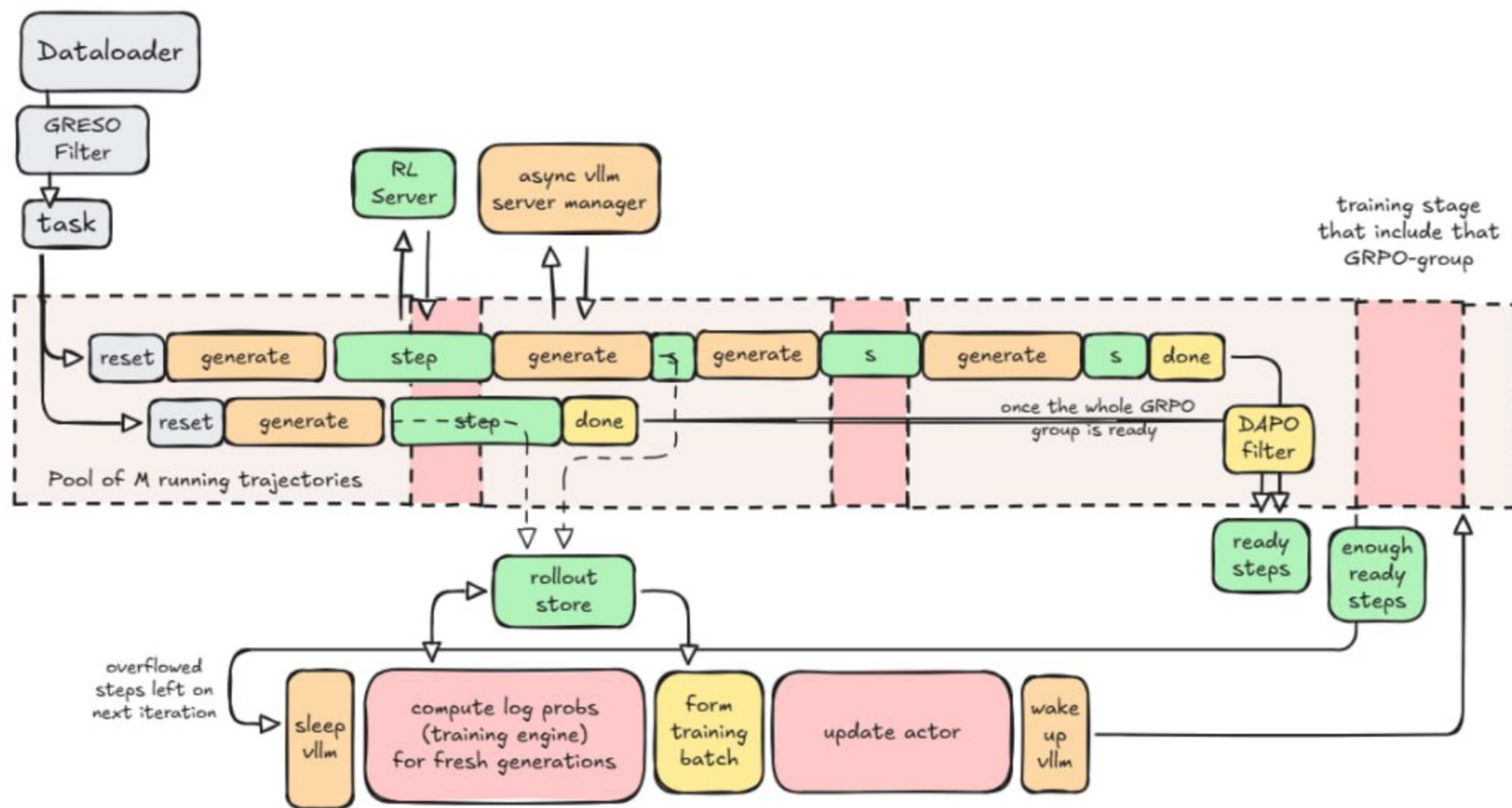
Агентный rl фреймворк

Async Policy



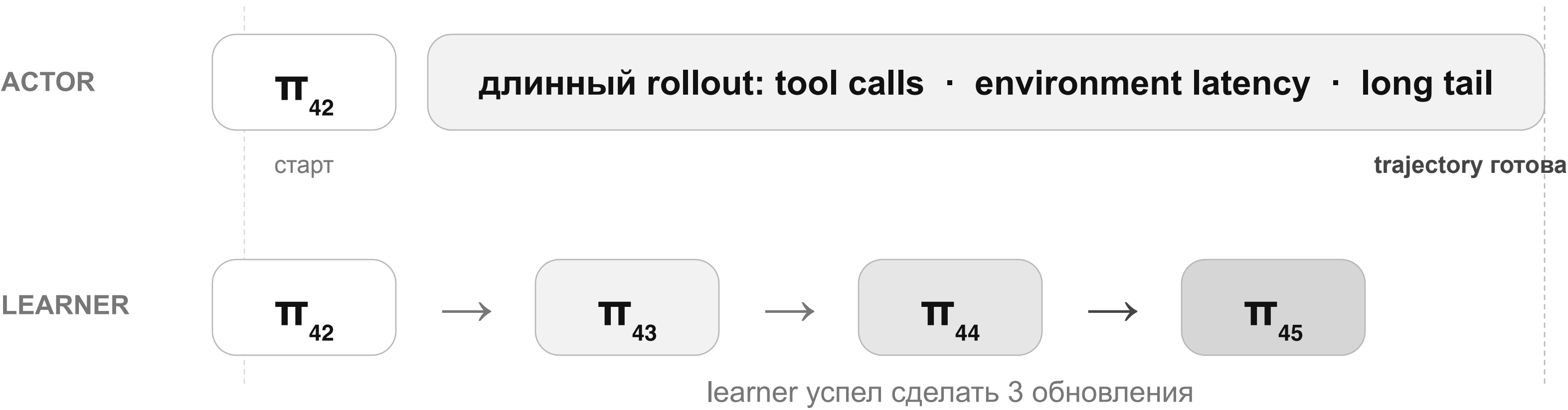
Агентный rl фреймворк

Async Policy



Пока актер думает, policy уже устарела

Async RL убирает простои — но превращает rollout-данные в off-policy



$$S = 3$$

policy updates

version(current) – version(behavior)

behavior policy = π_{42}

current policy = π_{45}

Считаем в версиях модели,
а не в секундах

ПОЧЕМУ В АГЕНТНОМ RL ХУЖЕ

длинные траектории + tool latency + stragglers

policy gap растёт → ratios уходят от 1 → PPO/GRPO клипает больше токенов → effective batch уменьшается

ВЫВОД

Staleness нужно ограничивать, а не обнулять

$$S \leq \eta$$

0 — синхронно и медленно

bounded

∞ — быстро и нестабильно

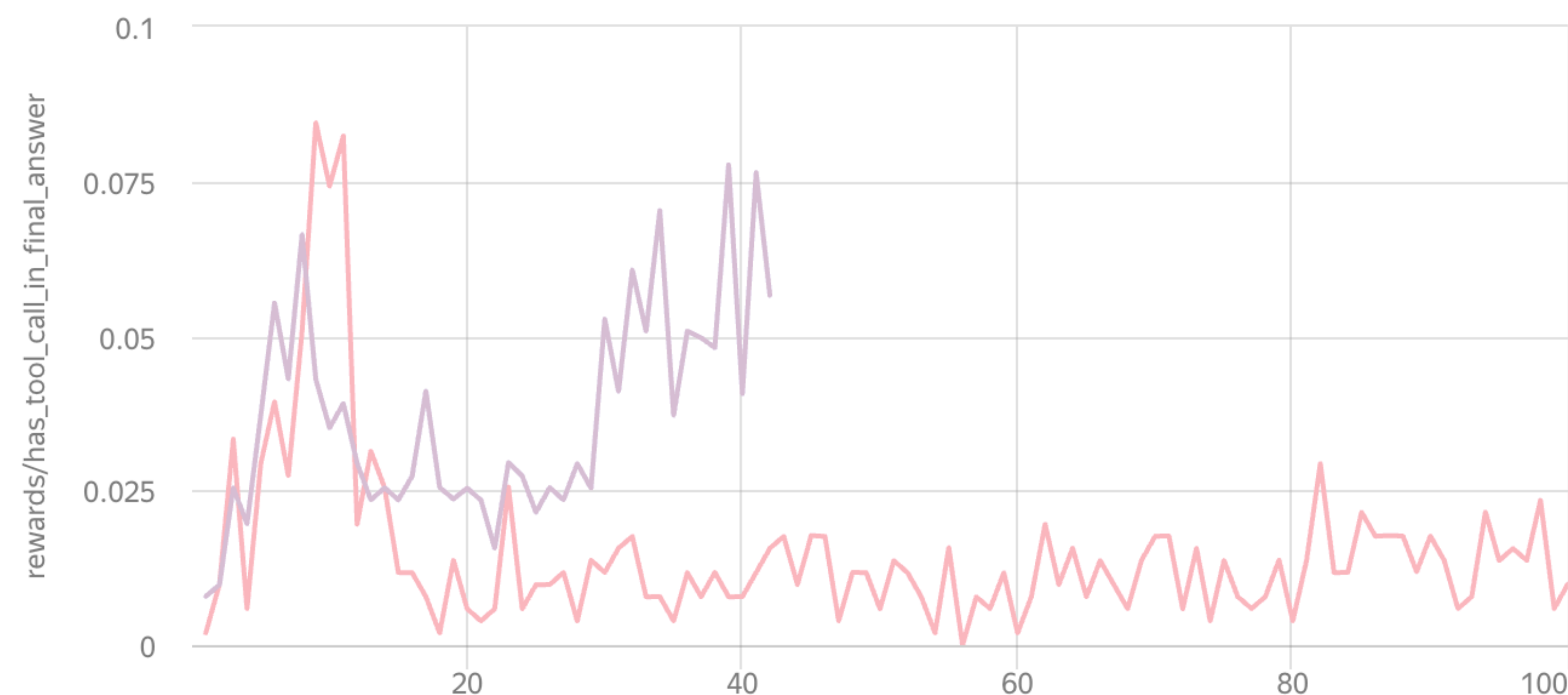
Чиним harness

Коллапс невалидных тул колов

Меняем лосс

- Модель сфейлила на терне -> остановка траектории и плохой реворд
- Проблема: сломанные траектории имеют мало токенов по которым течет лосс
- По дефолту везде агрегация лосс `batch_mean`, все токены в баче равновесные
- Делаем `seq_mean_token_mean` агрегацию

rewards/has_tool_call_in_final_answer



Первые развалы

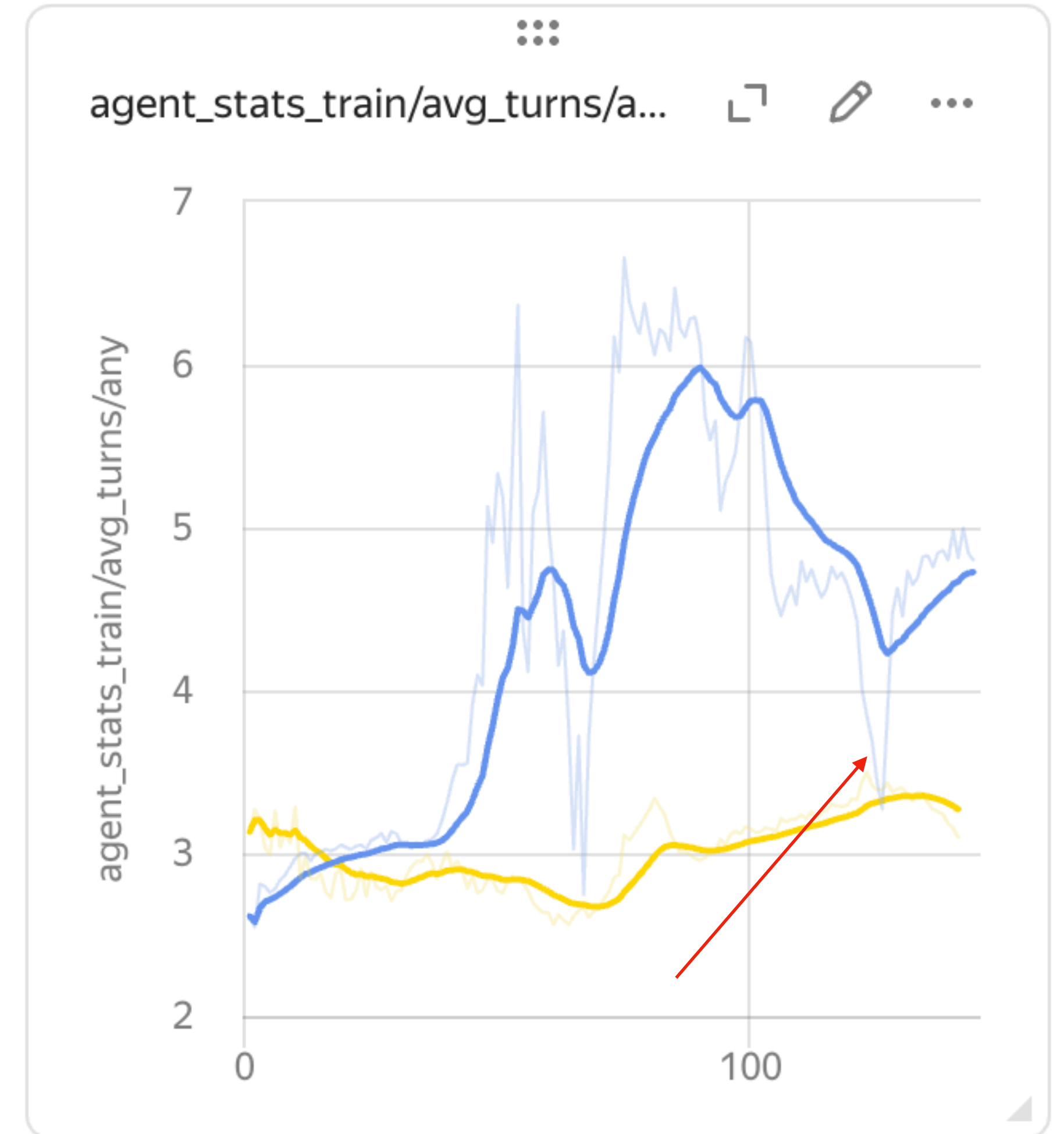
Правда ли это хорошее решение проблемы ?

- Даже с таким фиксом кол-во ошибок не сводится к нулю
- Введем типизацию ошибок
- На каждую ошибку регламентируем фидбек передающийся модели на невалидные действия
- Добавляем мягкие штрафы за невалидные действия

Error ID	Failure Mode	Trigger Mechanism	Protocol Response Role
Error 0	Wrong Tool Name	Model misspells or hallucinates a function name.	System
Error 1	Missing Parameter	Model omits a "required" field from the schema.	Tool
Error 2	Type Mismatch	Model provides an incompatible data type (e.g., <code>str</code> for <code>int</code>).	Tool
Error 3	Empty Value	Model provides a key but leaves the value as null or whitespace.	Tool
Error 4	Redundant Parameter	Model includes extra keys not defined in the tool interface.	Tool
Error 5	Format Violation	Malformed JSON (missing braces, unquoted keys).	System or Tool
Error 6	Prose Inclusion	Model adds natural language explanation inside the tool block.	System
Error 7	Wrong Count	Model calls an incorrect number of tools for parallel execution.	System

Переполнение контекста

- При обучении на сложных задачах у агента есть соблазн как можно больше ходить в поиск
- Но модель сама не знает когда у нее кончается контекст
- Лечит все мягкий штраф за большое количество походов в поиск



Переполнение контекста

Алгоритм контекстного менеджера

Контекст заполнен на 90%

- отбрасываем последнее `observation`
- добавляем: «*Инструменты больше не используй. Заверши ответ*»
- новый tool call получает сильный штраф в рл
- **агент выдаёт результат**

Было: агент продолжает вызывать инструменты → context overflow → траектория потеряна

Стало: последние 10% контекста зарезервированы под завершение

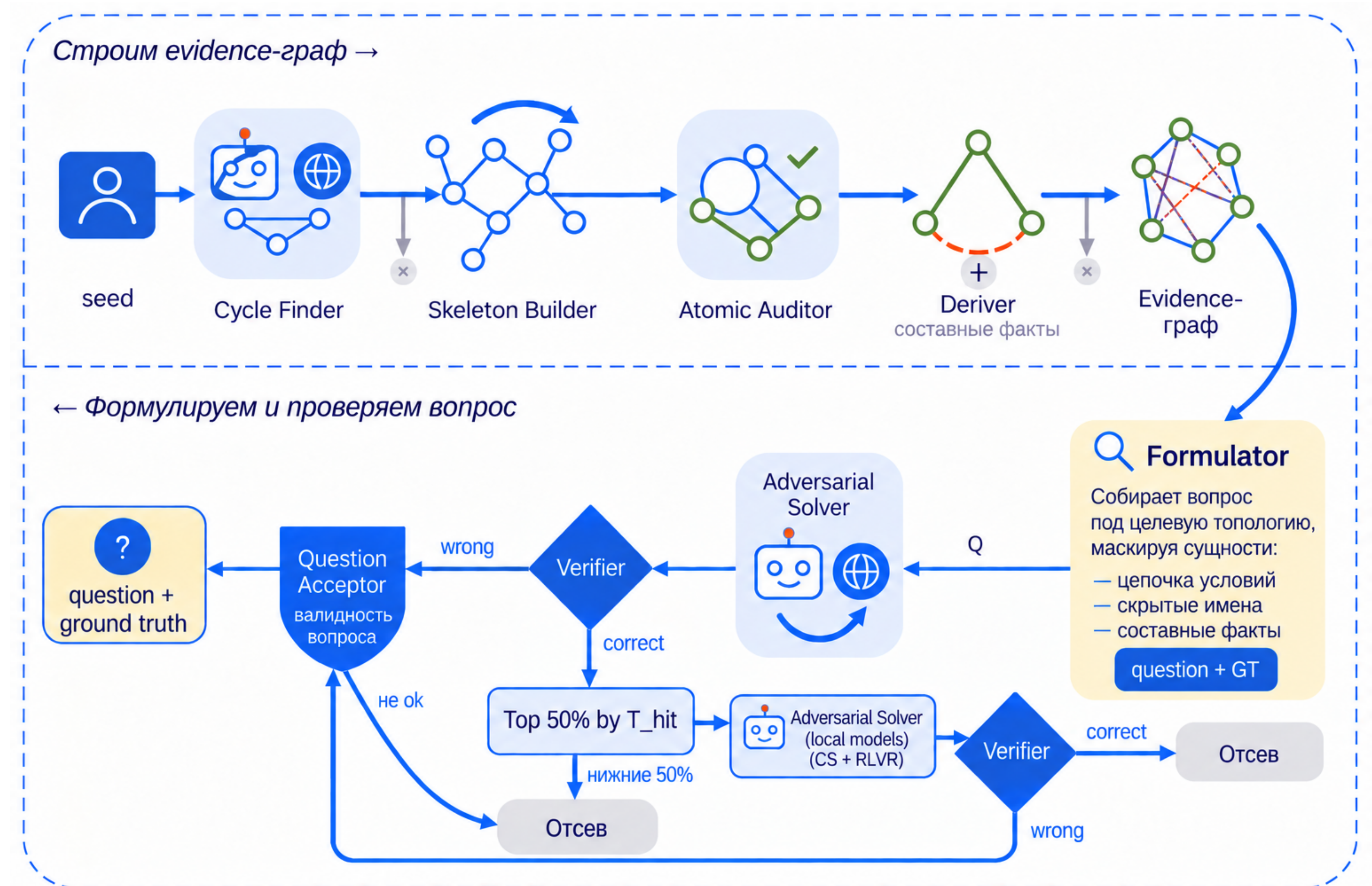
Сейчас пошли еще дальше, добавляем на каждом терне

“Использовано 65% контекста[80k из 120k]”

Учимся глубоко искать

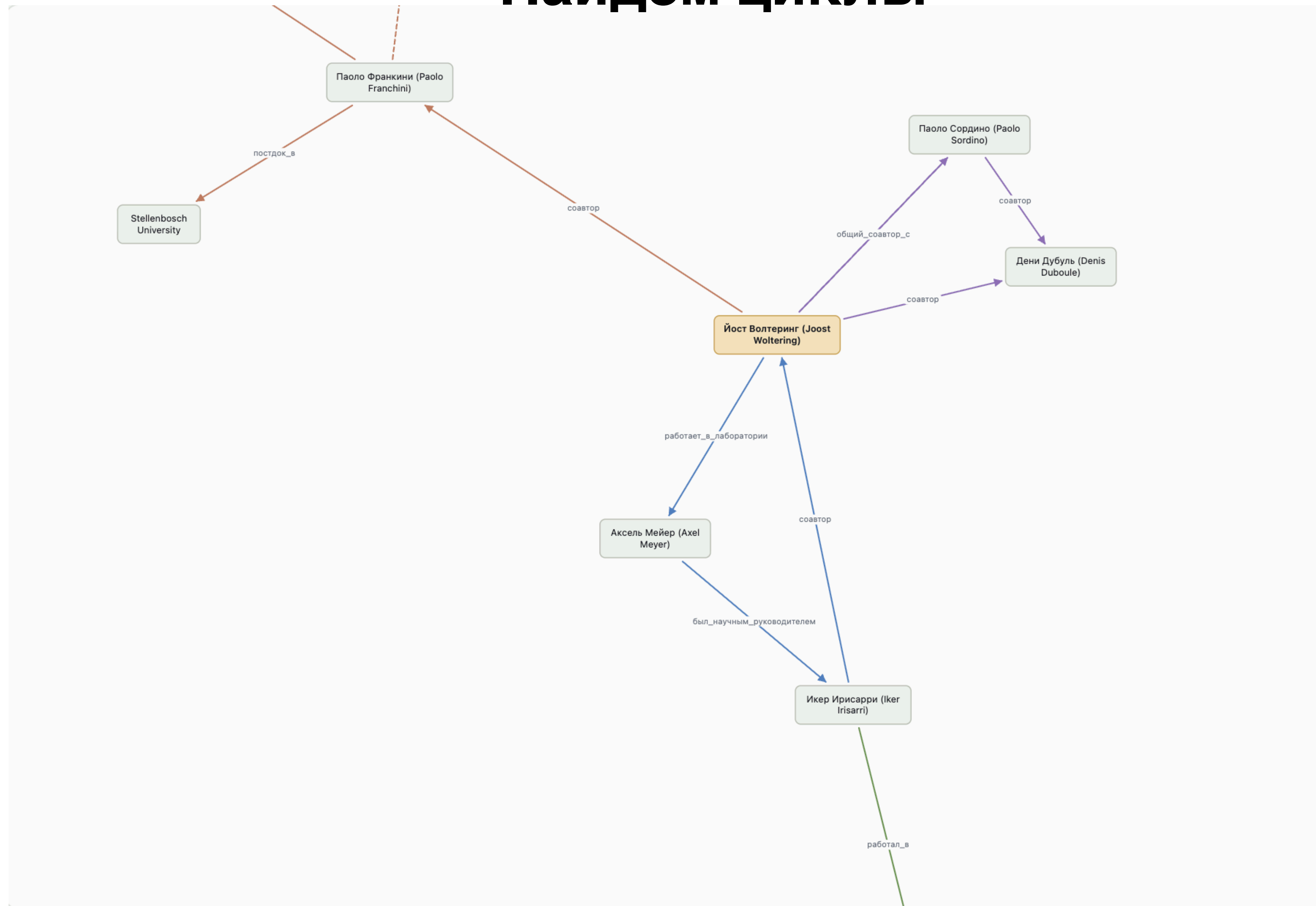
Как сделать сложный вопрос?

T_hit - номер терна в котором всплыл правильный ответ



Как сделать сложный вопрос ?

Найдем циклы



Как сделать сложный вопрос ?

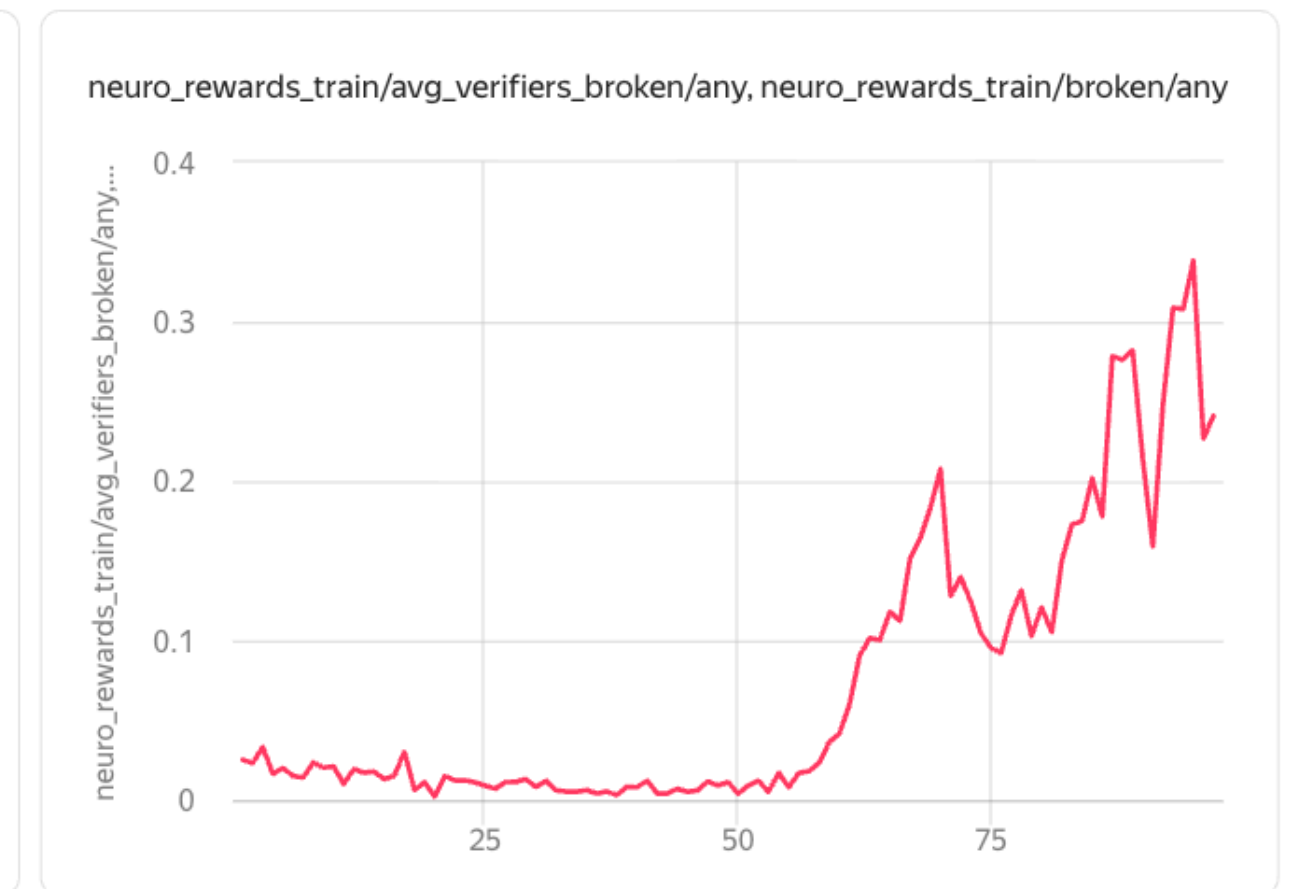
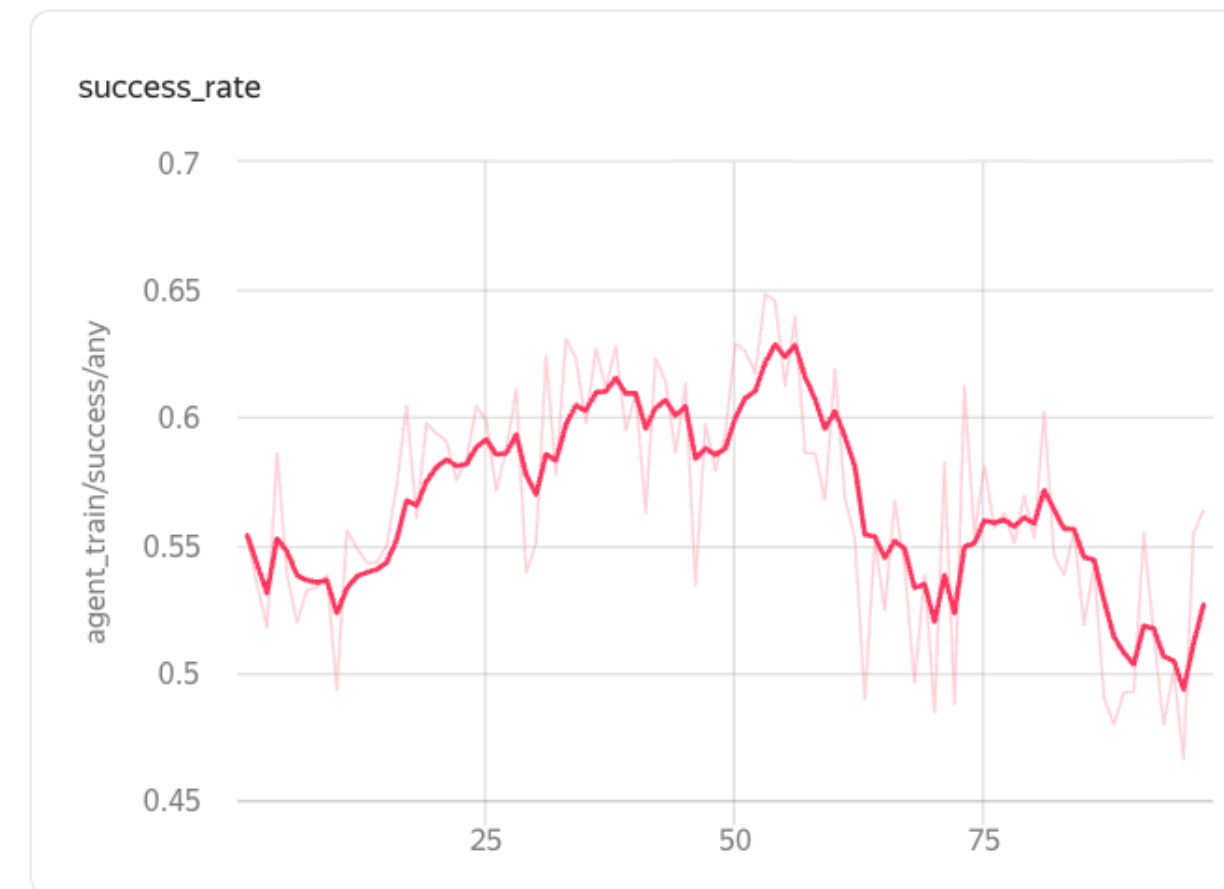
Биолог, опубликовавший в 2021 году исследование в авторитетном научном журнале, на момент публикации работал в лаборатории профессора, под руководством которого в 2010-х годах проходил постдок-стажировку его соавтор по другой публикации. Этот соавтор в начале 2020-х годов работал в одном из исследовательских институтов. Другой соавтор того же исследователя защитил докторскую диссертацию в том же году, в который один из его собственных соавторов получил премию в области сравнительной онкологии. Наконец, ещё один соавтор исследователя и он сам имеют общего соавтора, с которым каждый из них публиковался отдельно, и при этом сами также являются соавторами. Назовите исследовательский институт, в котором работал упомянутый соавтор-постдок.

Дифф между движками

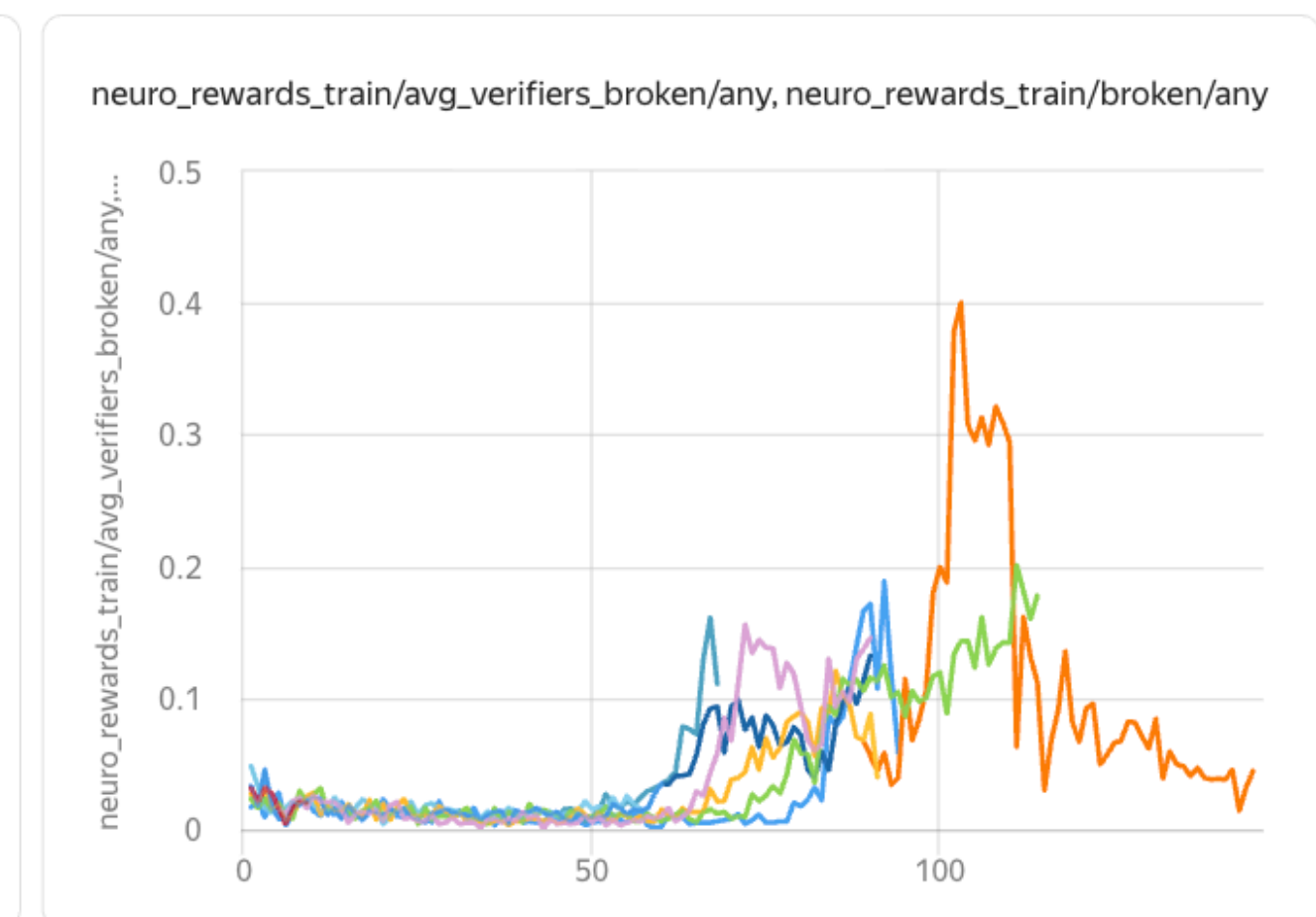
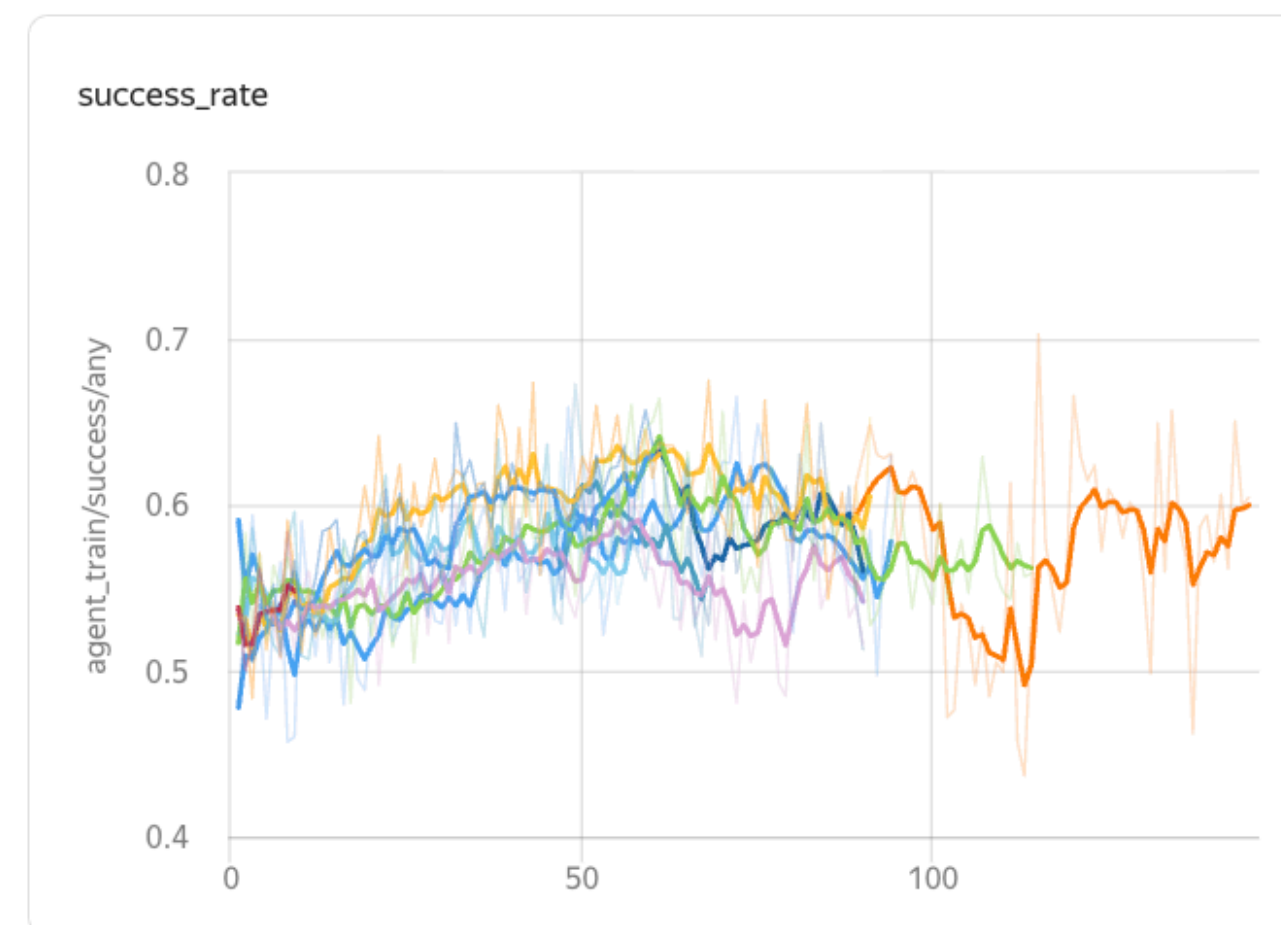
Взрывы обучения на сложных данных

Чиним reward

- Переехали в какой-то момент на MOE + новый фреймворк
- Все разнесло
- Начали делать попытки чинить реворд



Первый запуск



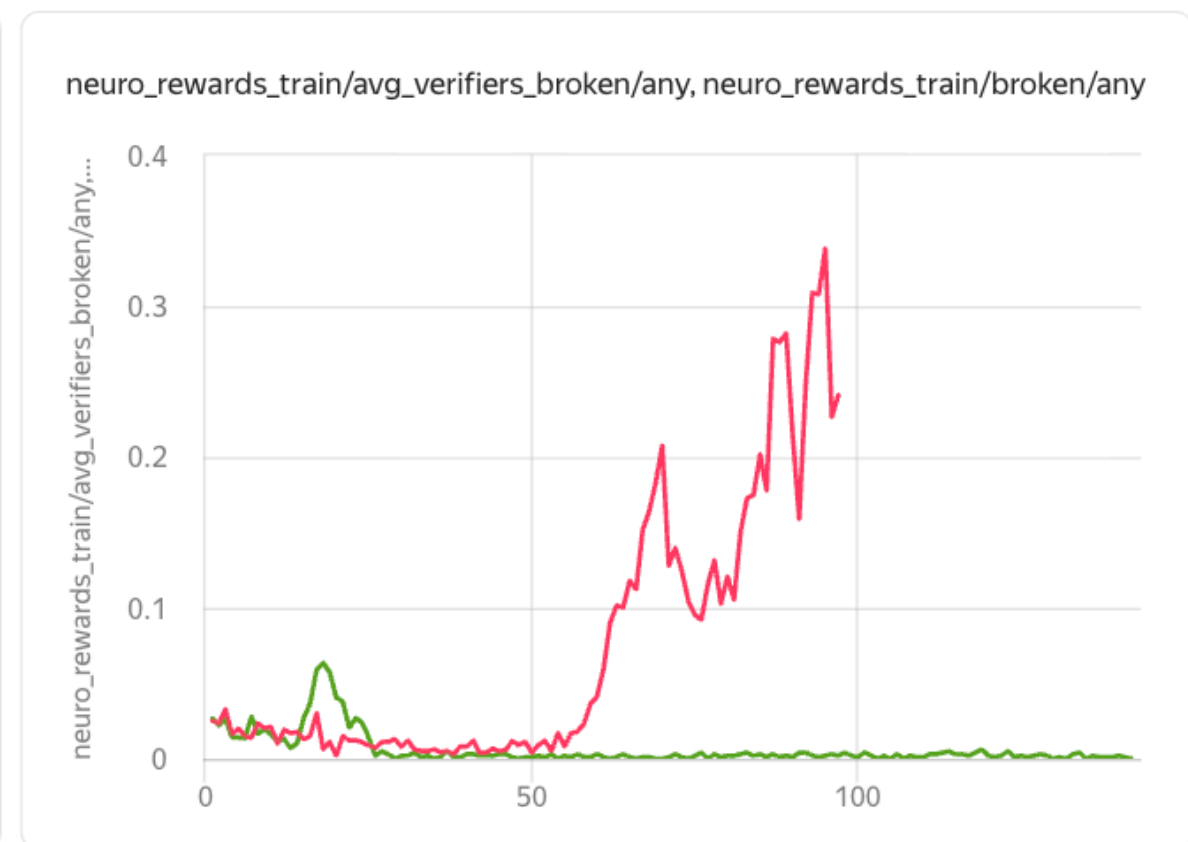
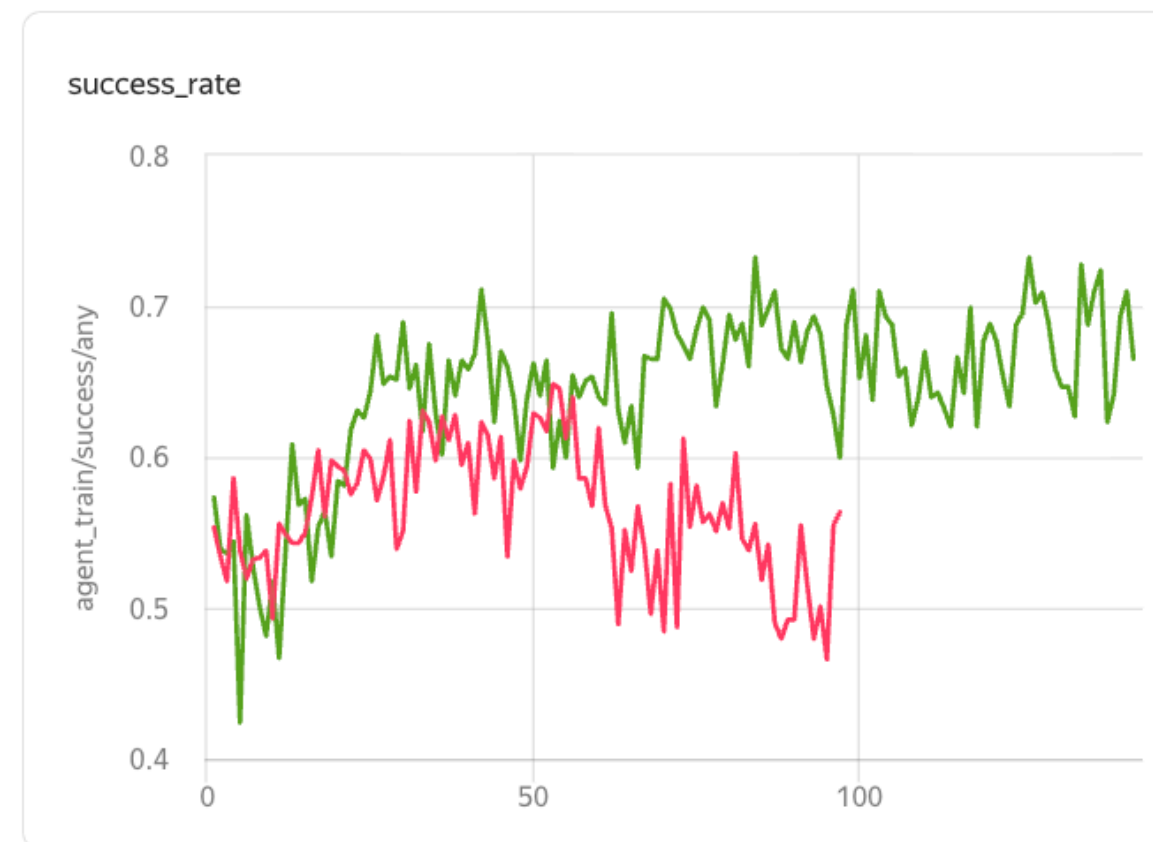
Попытка хачить реворд

Взрывы обучения на сложных данных

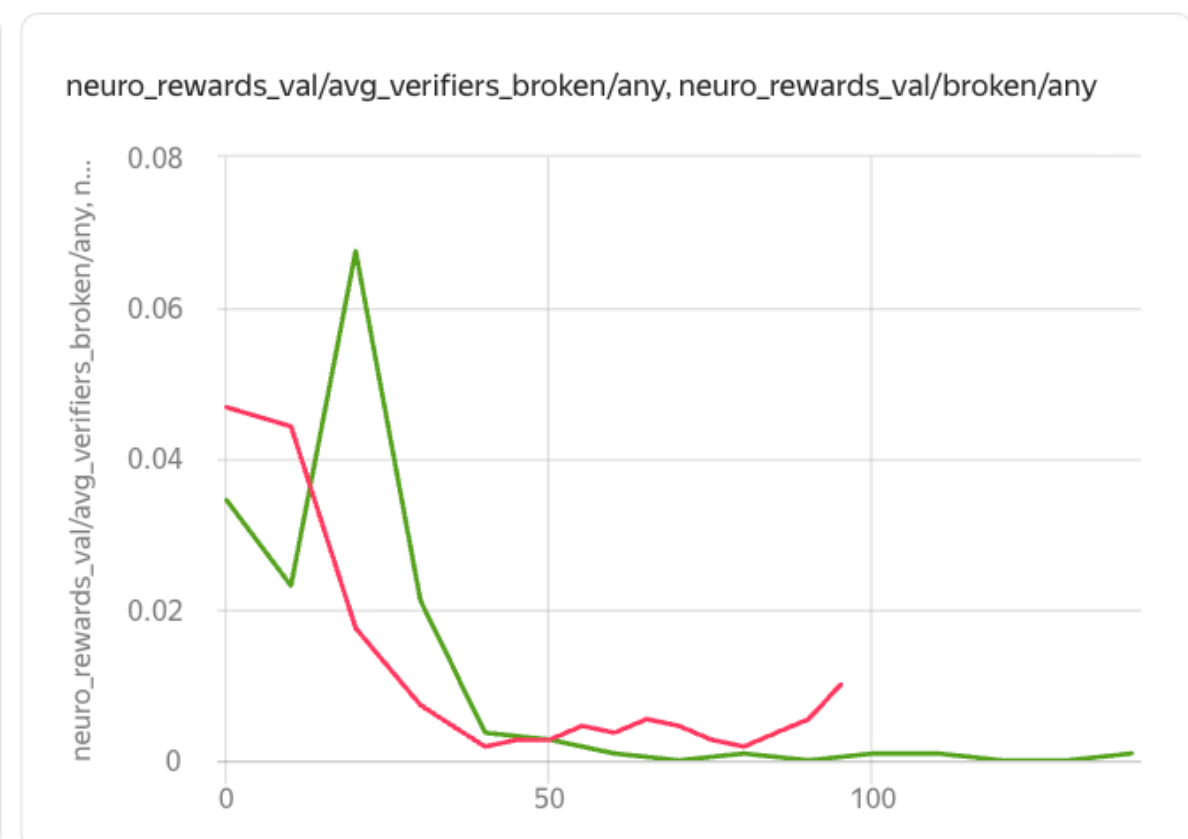
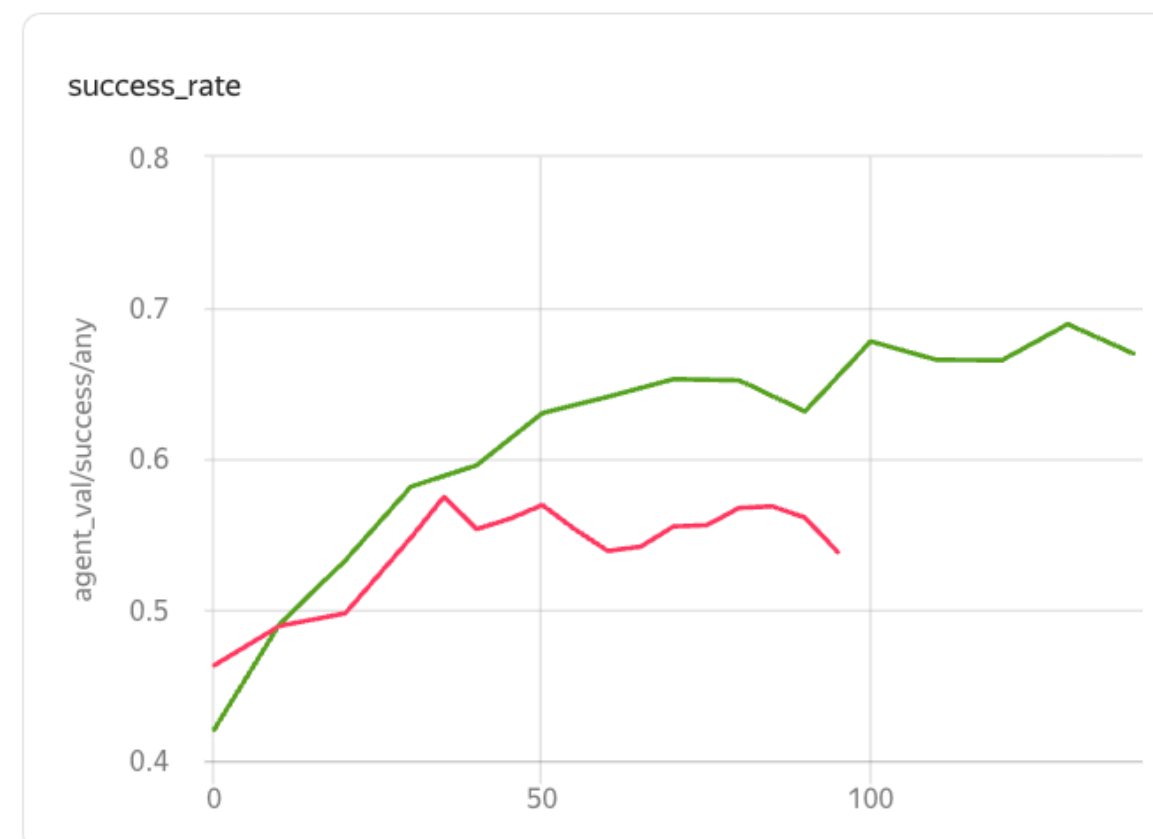
Давайте попробуем dense модель

- Пришлось долго копать тк при переезде на новый фреймворк поддерживали только megatron
- На dense модели внезапно все работает
- Проблема в МОЕ!

Train main 2

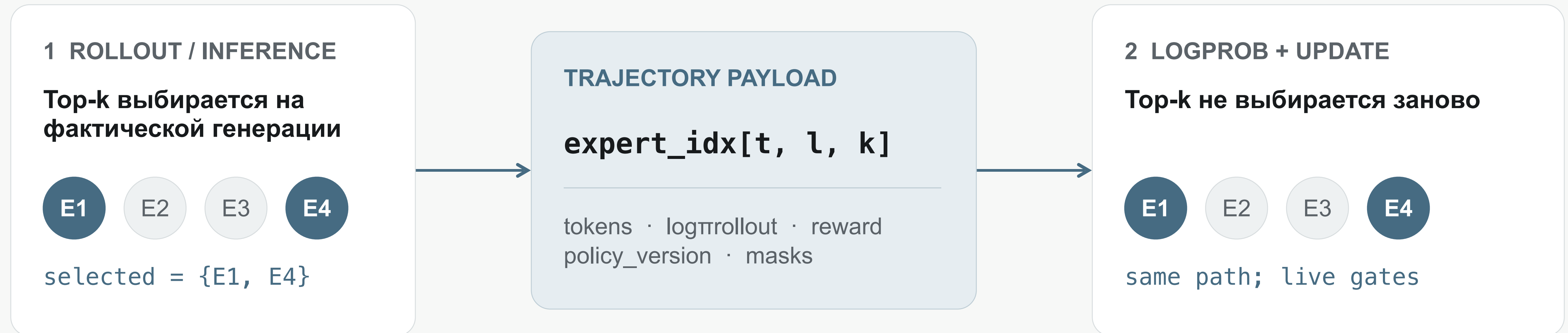


Val main 2



R3 переносит expert path из rollout в training

Rollout Routing Replay переносит дискретный top-k выбор экспертов вместе с траекторией; live router weights остаются обучаемыми.



ФИКСИРУЕМ ПУТЬ

Itrain := Irollout

СОХРАНЯЕМ GRADIENT FLOW

gtrain $\propto$ Irollout $\odot$ f(strain)

R3 СНИМАЕТ

Backend-induced routing mismatch — rollout и train logprobs считаются по одному expert path.

R3 НЕ СНИМАЕТ

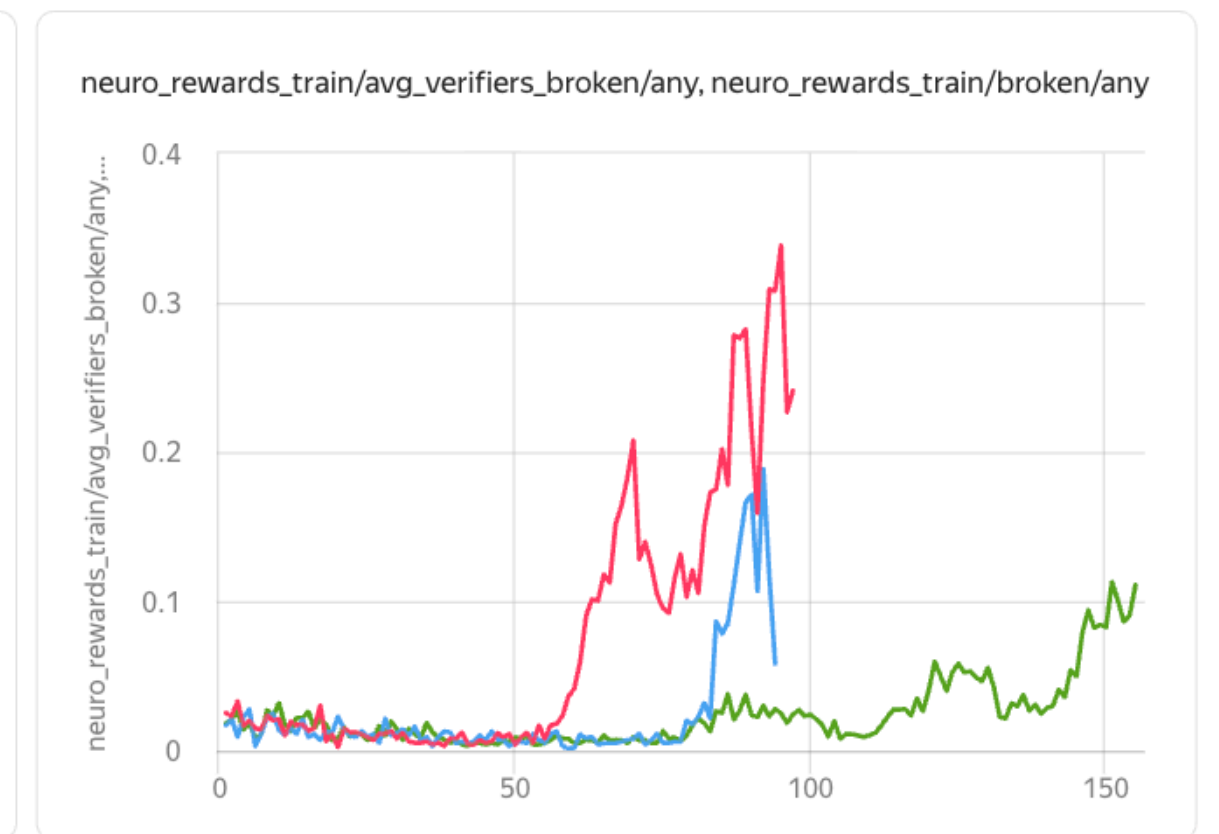
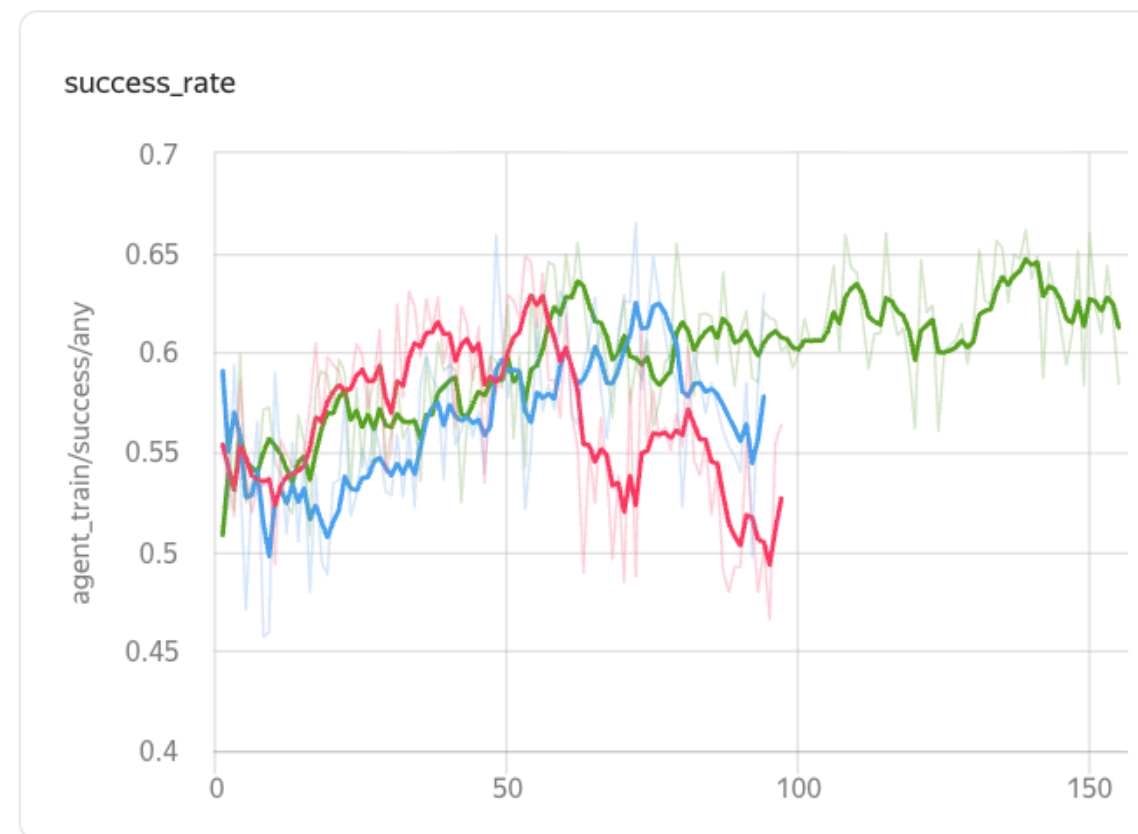
Policy lag / off-policy staleness в async RL — для них остаются отдельные trust-region и correction механизмы.

Взрывы обучения на сложных данных

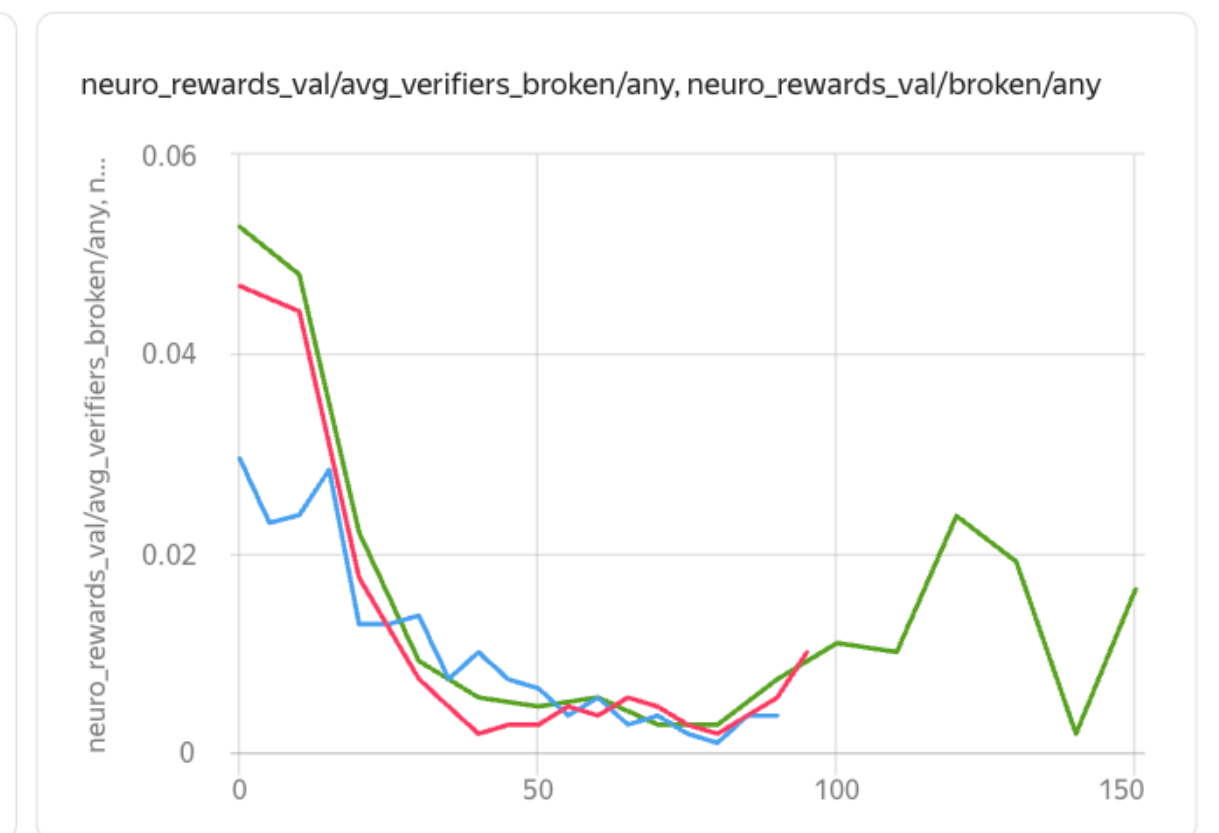
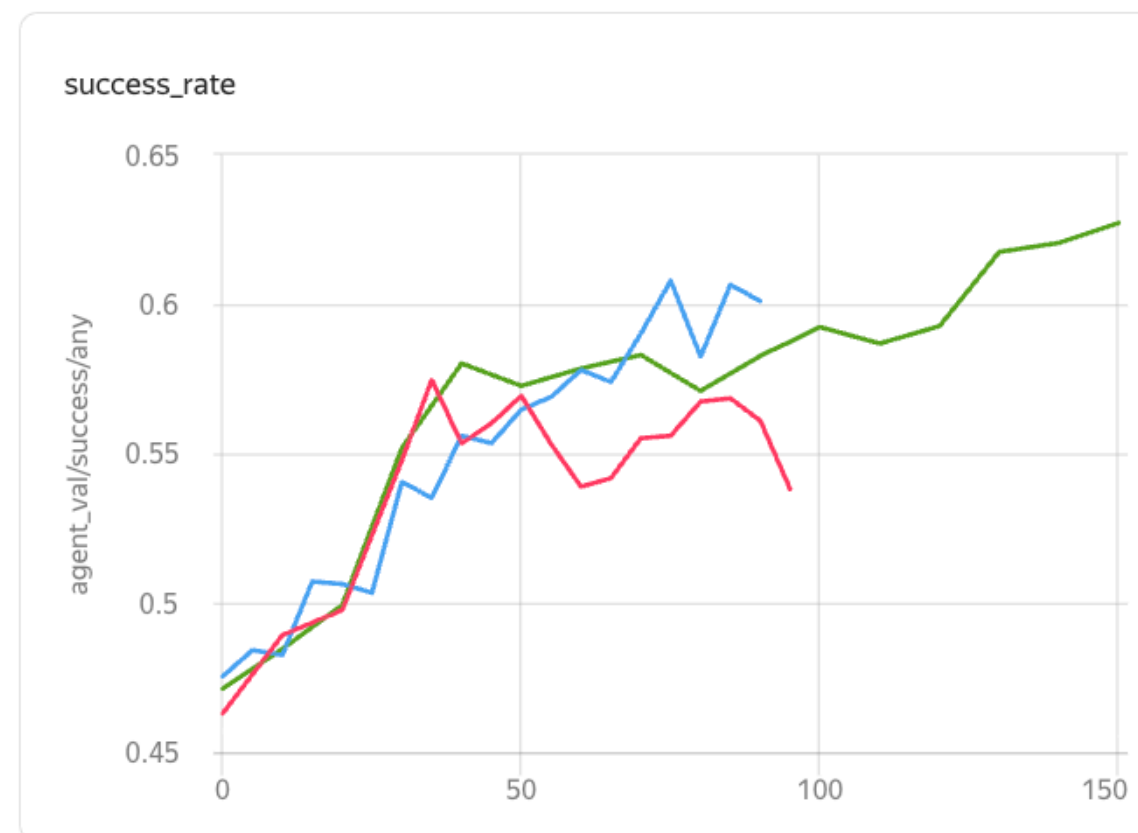
Давайте сделаем on-policy + r3

- Staleness = 0
- R3 включаем

Train main 2



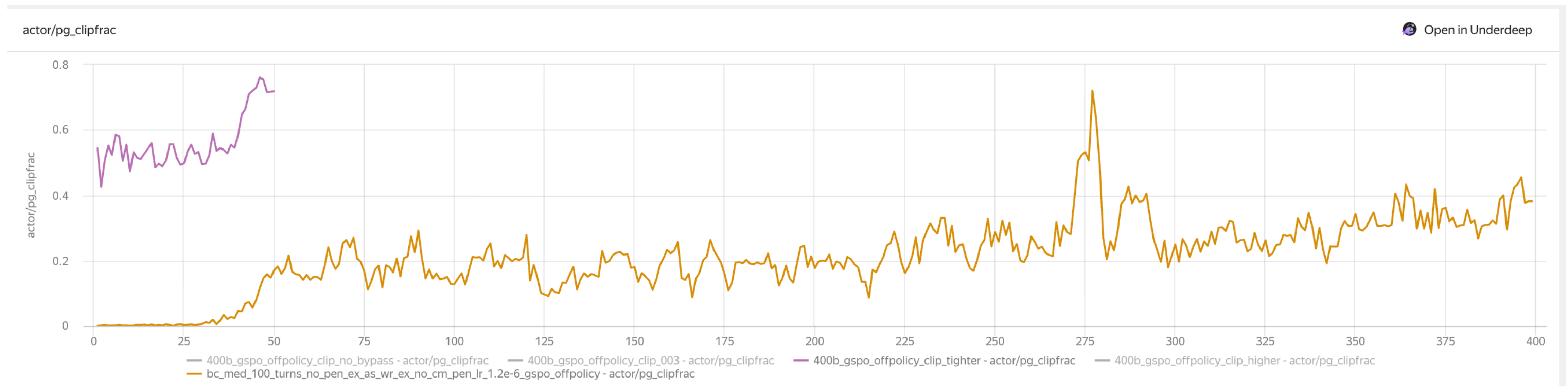
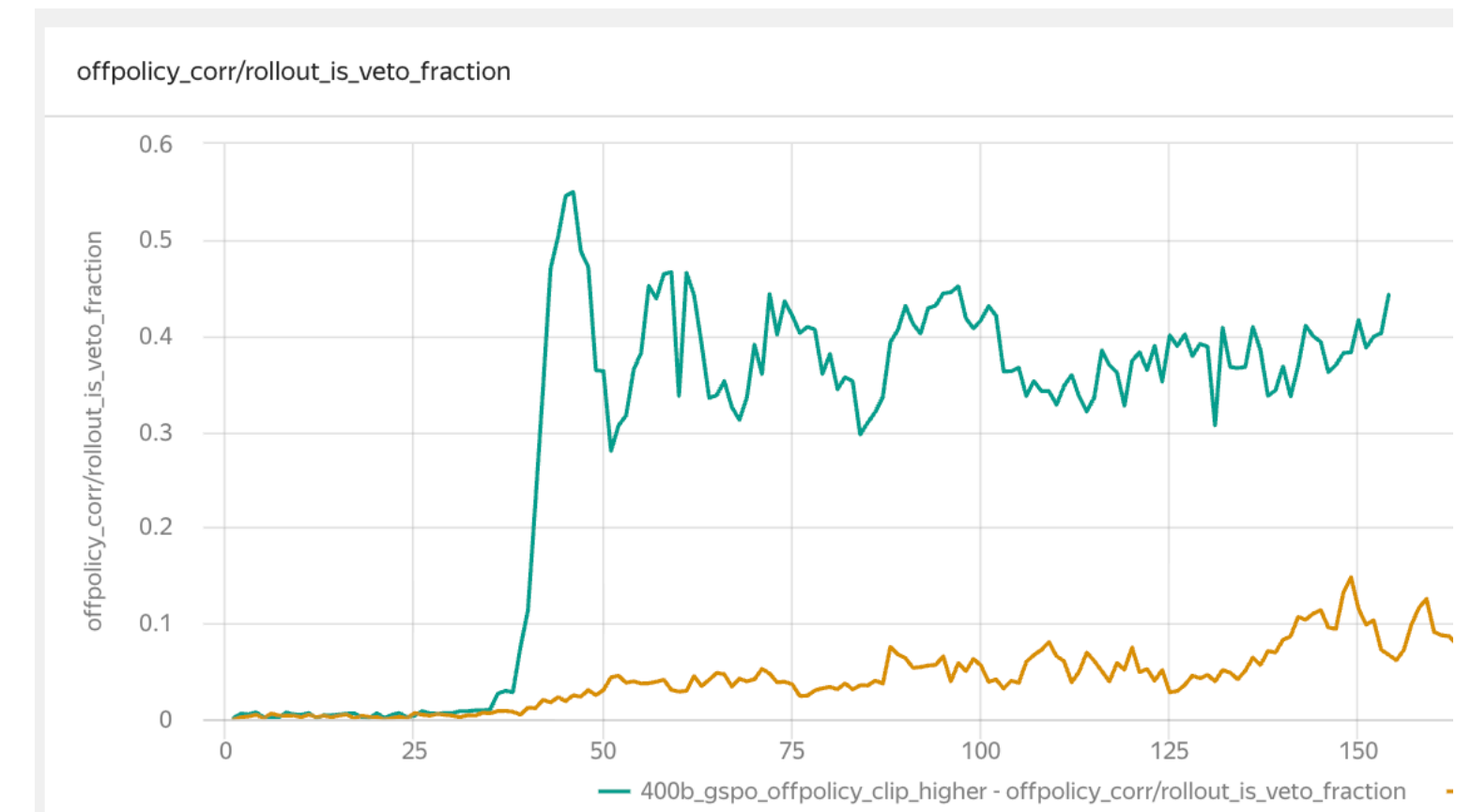
Val main 2



Скейлим модель

R3 наносит ответный удар

- У 400В модели 512 экспертов а не 256, ошибка роутинга чаще
- R3 не работает наивно: R3 стейт весит 460 GIB



Взрывы обучения на сложных данных

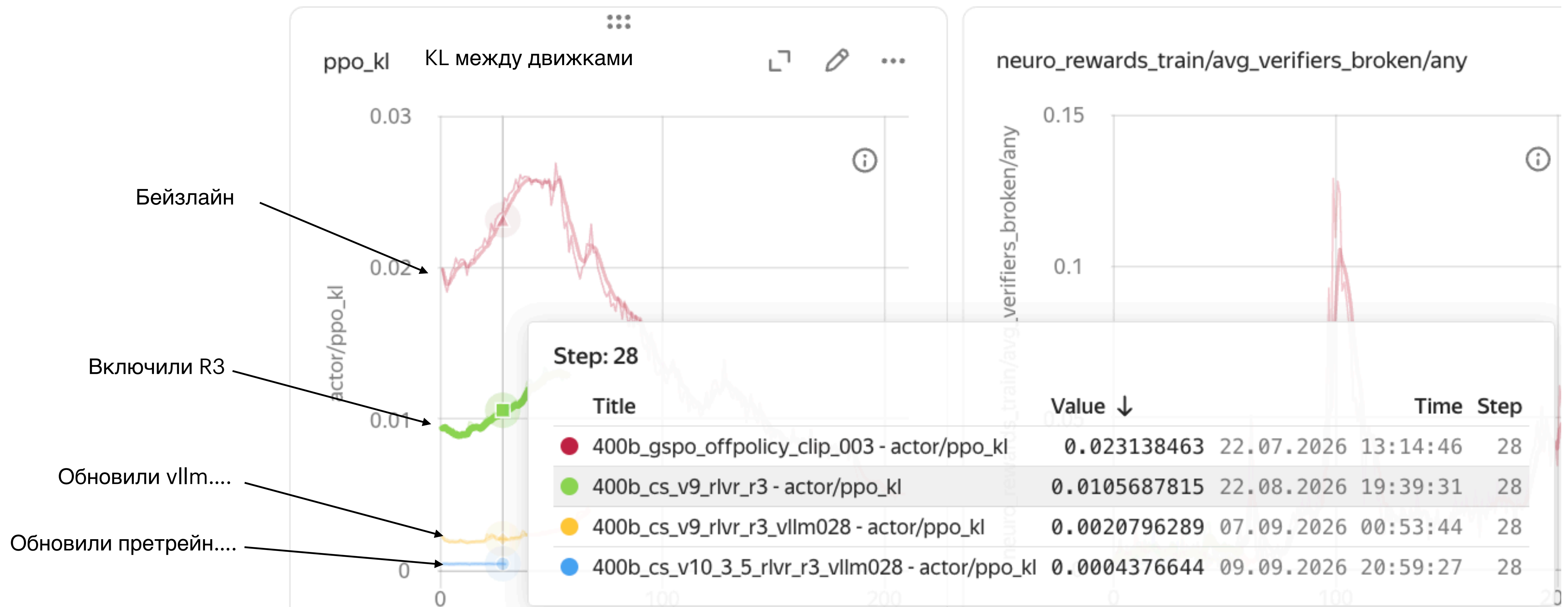
А что по staleness ?

- Сделаем больше staleness
- Качество хуже -> надо ресерчить алгоритмы дальше
- **Проблемы с памятью!**
- Больше staleness -> больше параллельных генераций
- Для 2048 параллельных генераций длины 100к KV Cache + DeltaNet states - 3.9TiB.
Если обучение на 64 карты то **62 Gb** на карту
- Надо делать квантование kv кэшей + оффлоадинг
- **Что такое R3 со staleness ???**



Неочевидные вещи

У движка rollout и actor разные кернелы



**А как понять что модели
плохо ?**

Вспоминаем на что учимся

Reinforce

$$\mathcal{L}_{\text{REINFORCE}}(\theta) = -\frac{1}{B} \sum_{i=1}^B \text{stopgrad}(A_i) \log \pi_{\theta}(y_i | x_i)$$

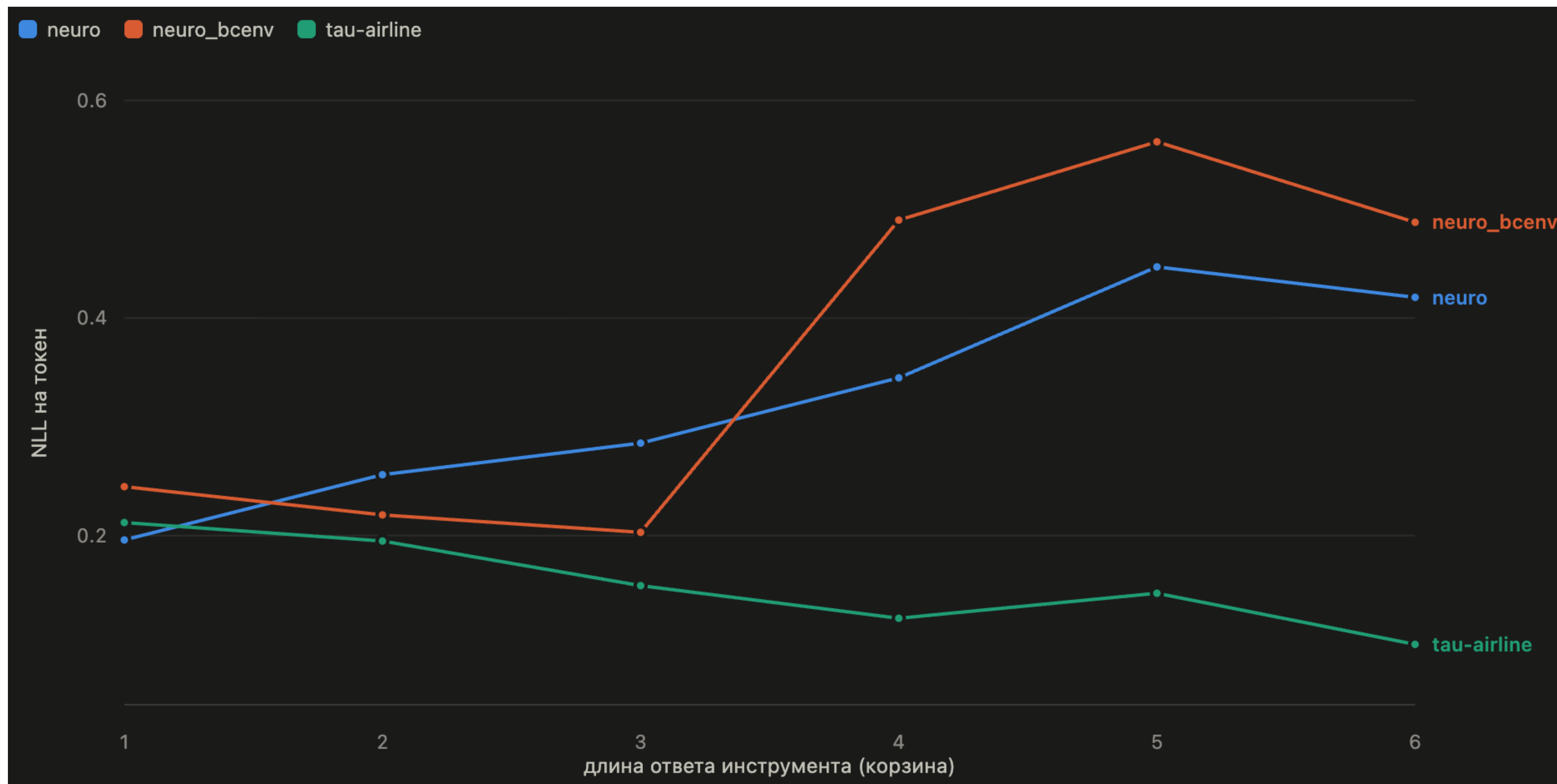
$$A_i = R(x_i, y_i) - b(x_i)$$

Хотим чтоб вероятность “хороших” цепочек росла, но:

- могут быть похожие цепочки в баче с очень разным ревордом
- реворд очень разреженный
- в некоторых средах результат тула это OOD который смущает модель

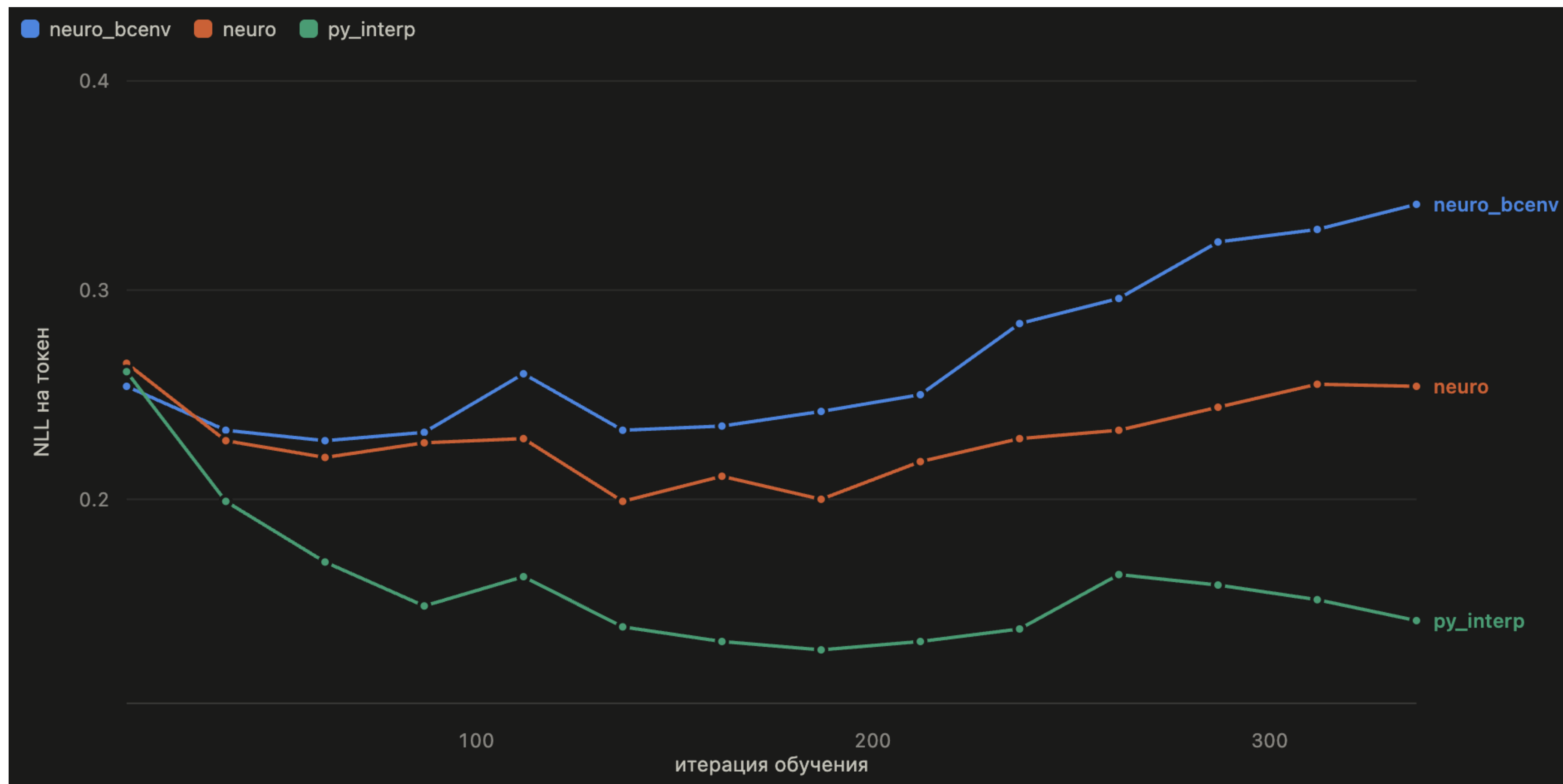
Вспоминаем на что учимся

Смотрим на вероятность траектории



Вспоминаем на что учимся

Более плотный реворд чуть-чуть улучшает ситуацию

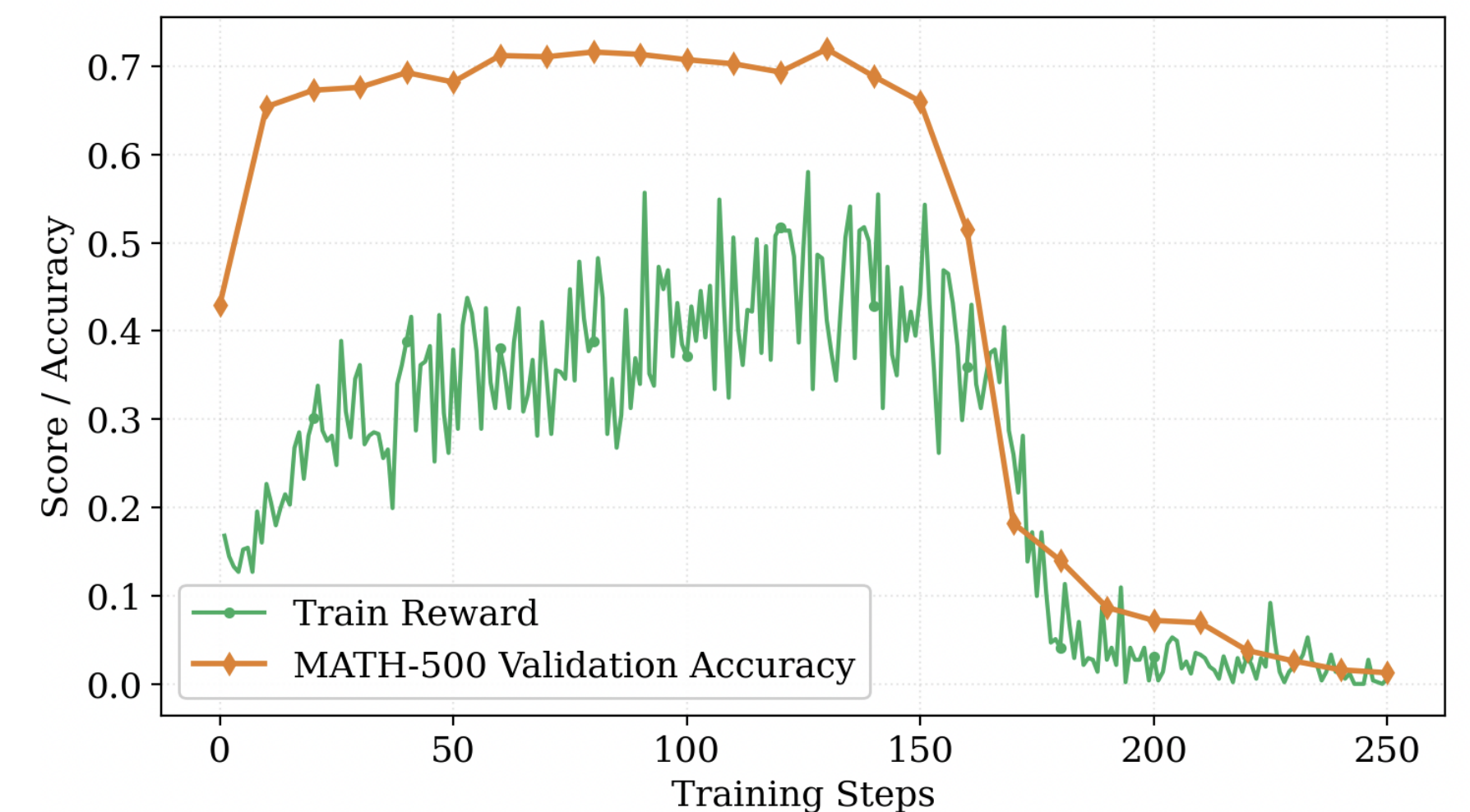
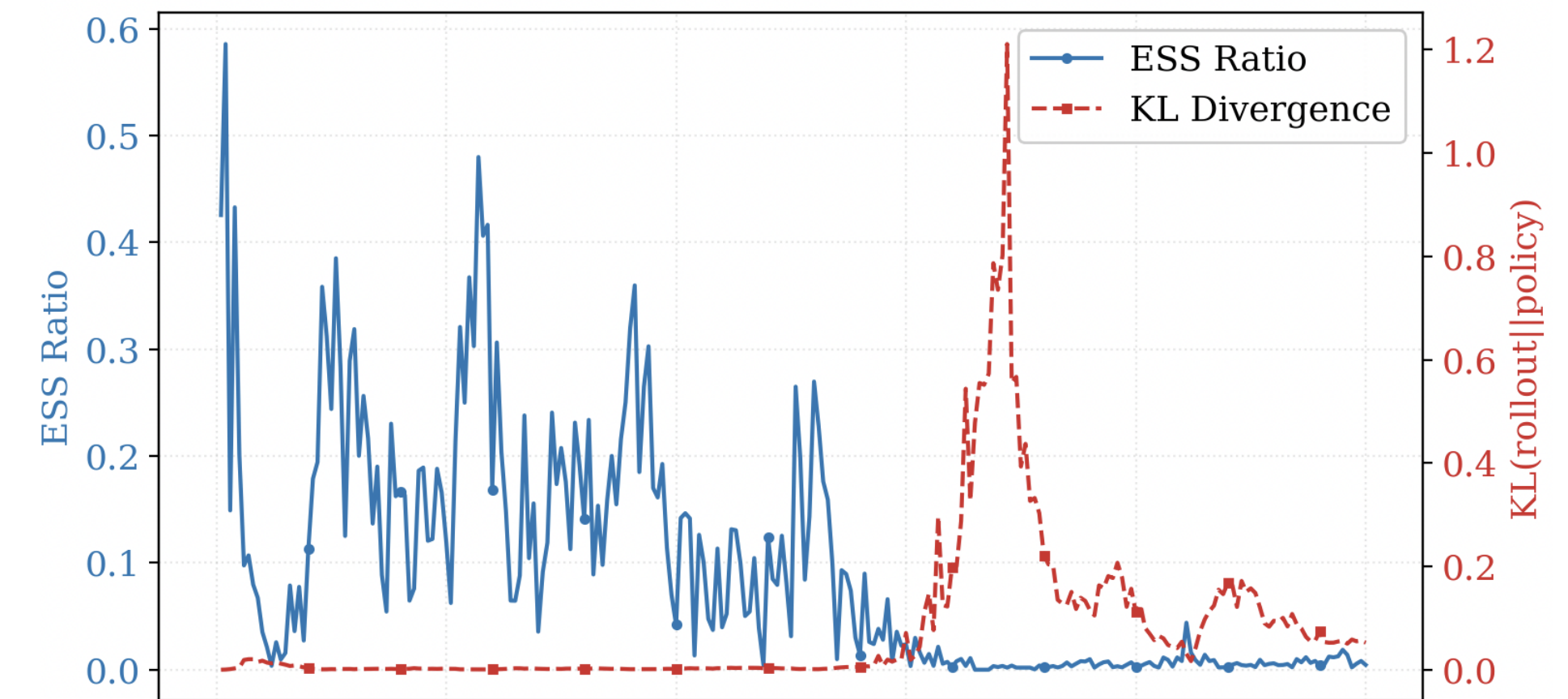


Лечим нестабильность ДВИЖКОВ

ESS

$$w_i = \frac{\pi_{\theta}(y_i \mid x_i)}{\mu(y_i \mid x_i)} = \prod_{t=1}^{T_i} \frac{\pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})}{\mu(y_{i,t} \mid x_i, y_{i,<t})}$$
$$\text{ESS} = \frac{\left(\sum_{i=1}^B w_i\right)^2}{\sum_{i=1}^B w_i^2}, \quad \rho_{\text{ESS}} = \frac{\text{ESS}}{B}$$

Math-500 with Qwen2.5-7B Base - 10 Steps Off-Policy



Лечим нестабильность движков

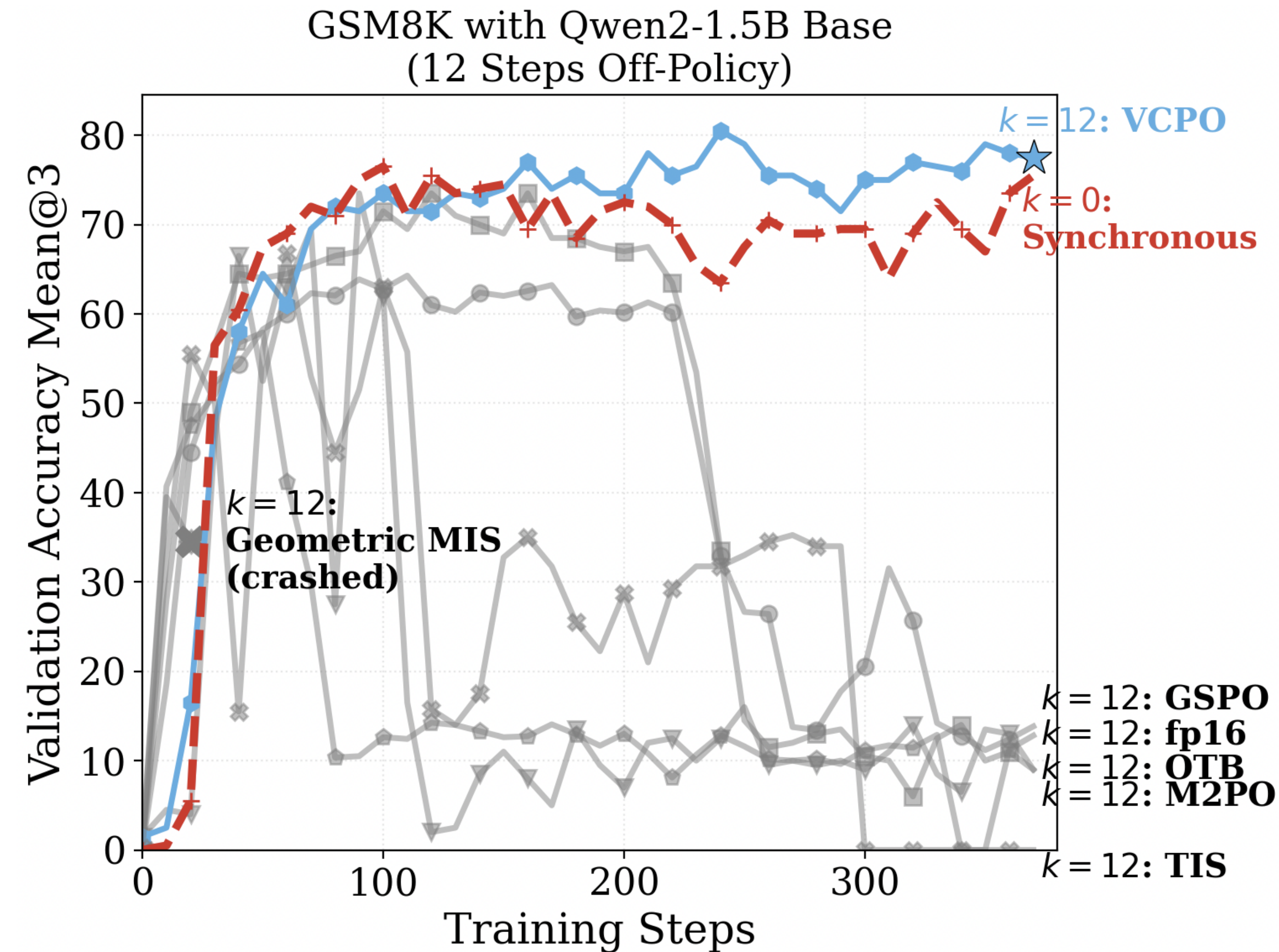
GSPO

$$r_{i,t} = \frac{\pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})}{\pi_{\text{old}}(y_{i,t} \mid x_i, y_{i,<t})}, \quad w_i = \prod_{t=1}^{T_i} r_{i,t}$$

$$s_i = w_i^{1/T_i} = \exp\left(\frac{1}{T_i} \sum_{t=1}^{T_i} \log r_{i,t}\right)$$

$$g_i^{\text{GRPO}} = \frac{A_i}{T_i} \sum_{t=1}^{T_i} r_{i,t} \nabla_{\theta} \log \pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})$$

$$g_i^{\text{GSPO}} = \frac{A_i s_i}{T_i} \sum_{t=1}^{T_i} \nabla_{\theta} \log \pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})$$



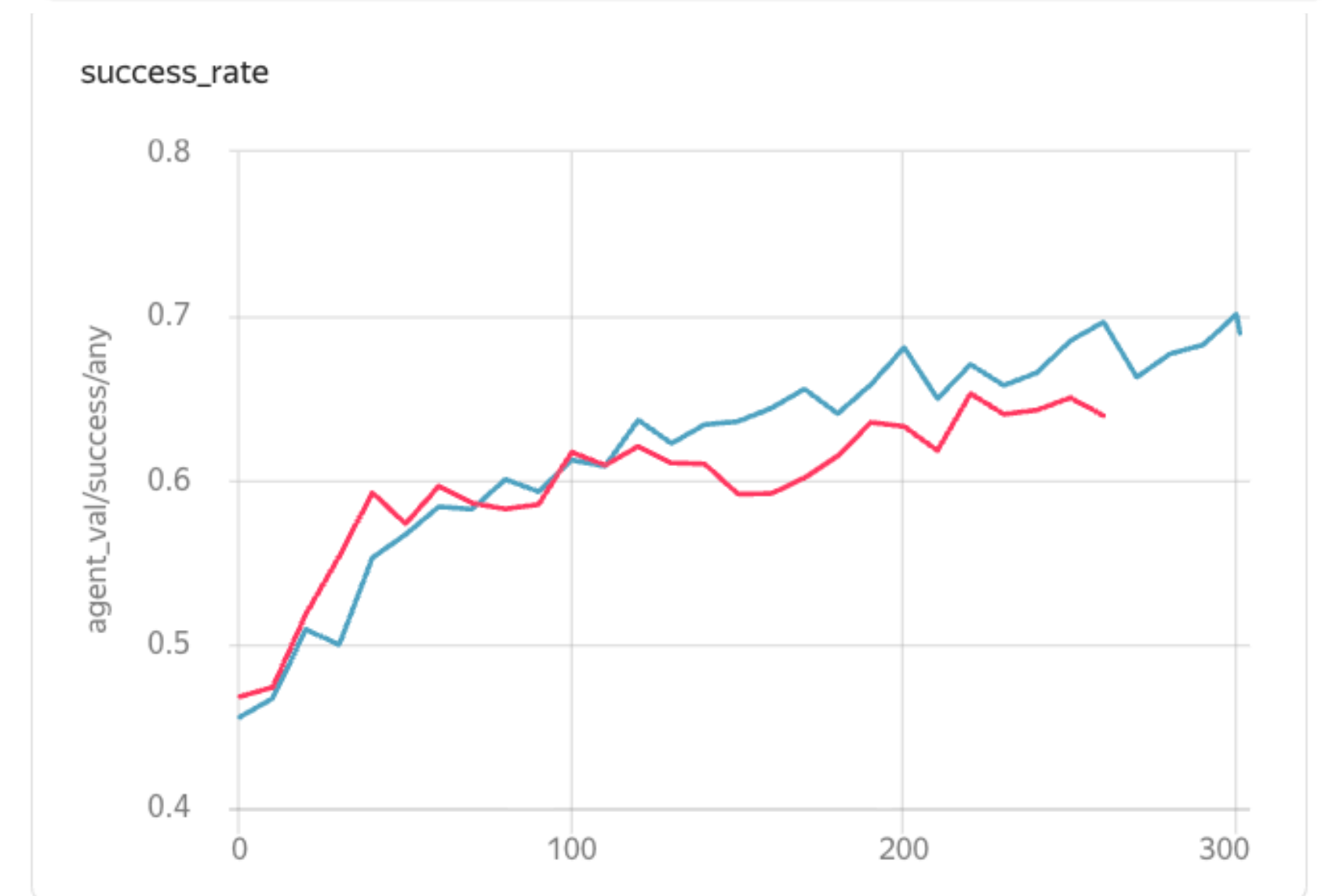
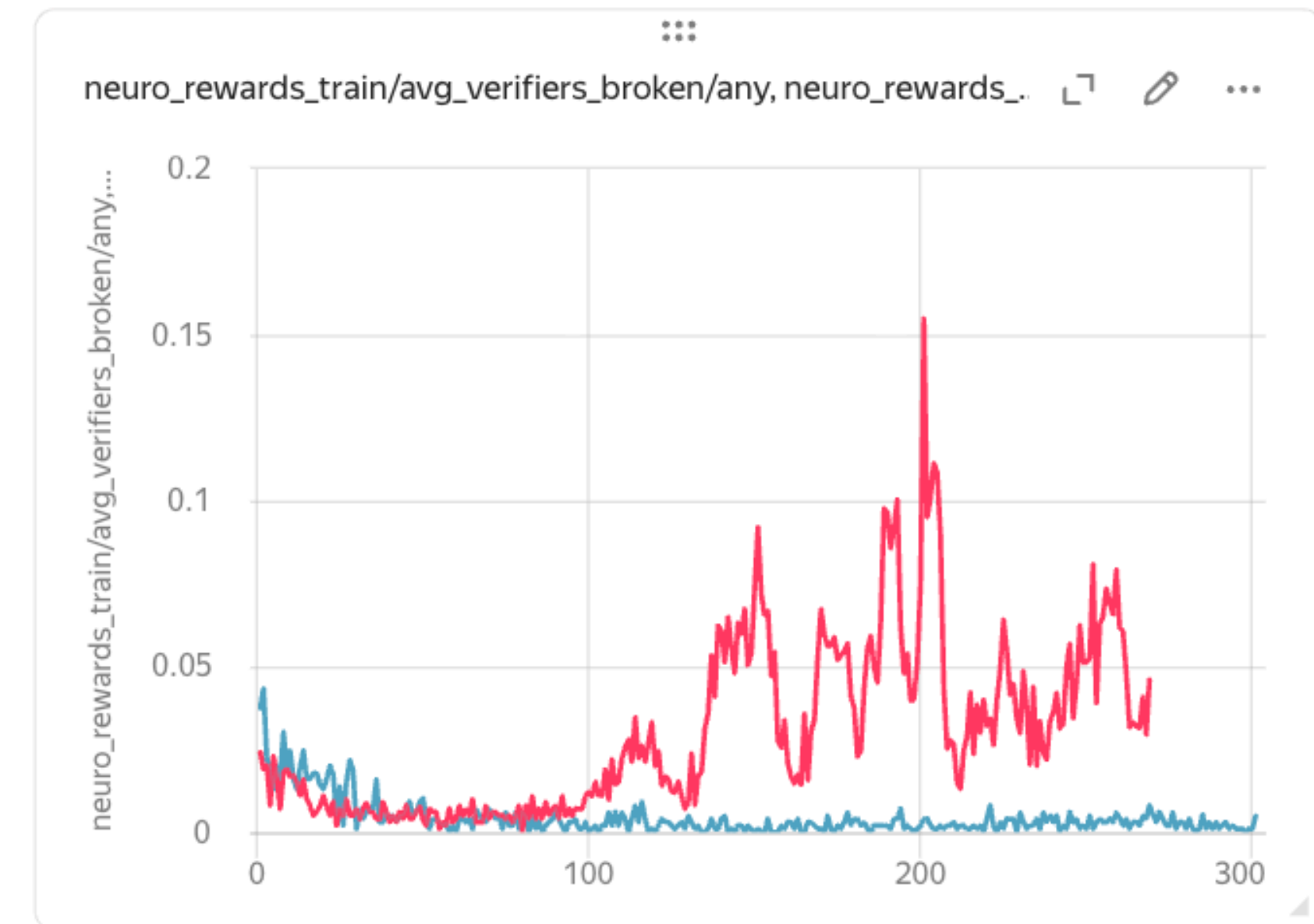
Взрывы обучения на сложных данных

GSPO

- Включаем GSPO
- Staleness держим ~0
- Все лечится
- Почти в 2 раза быстрее чем onpolicy + r3
- **Проблема: хуже на математике и коде**

3a35b_neuro_old_hp_offpolicy_gspo	3a35b_neuro_old_hp_r3_oc12
793.313828	1408.644888 +615.331059

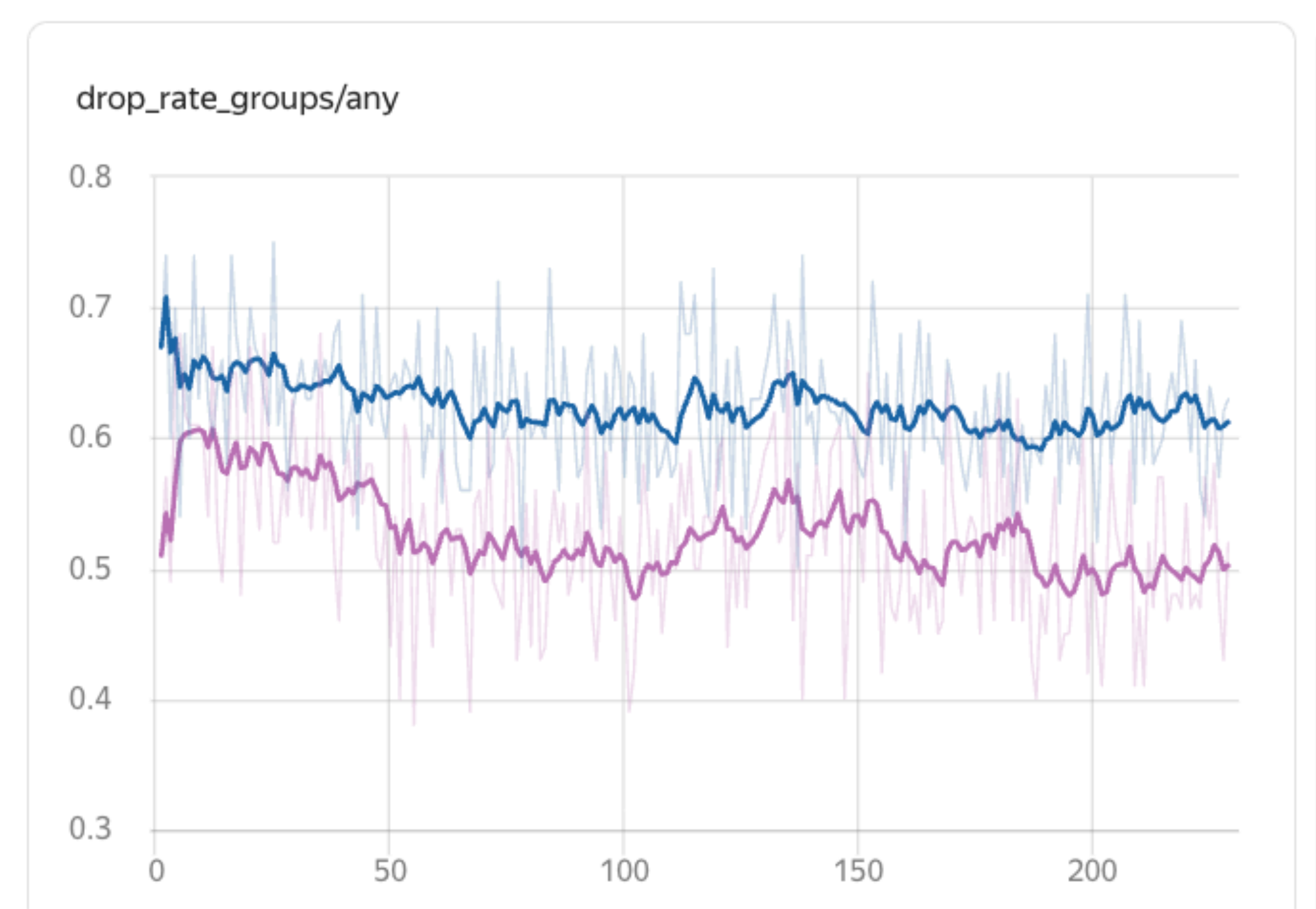
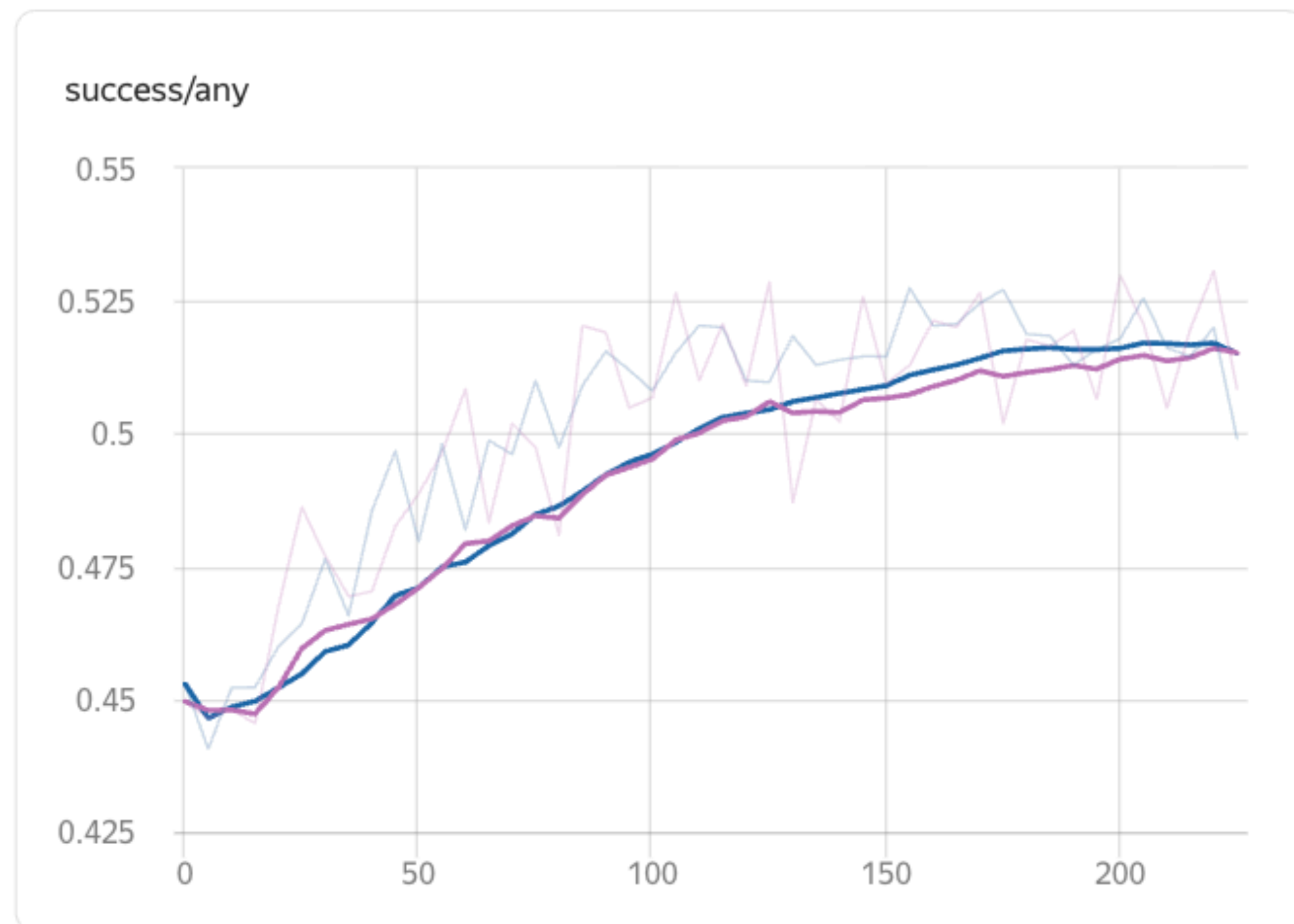
Время итерации



Эксп в который мы сильно верили

Dynamic sampling

- Дропаем группы с решаемостью $<15\%$ или $>85\%$ на каждом шаге обучения
- Эксп бежал ~10 дней, 0 профита
- Сейчас фильтруем перед обучением то что инишиал решает 0/10

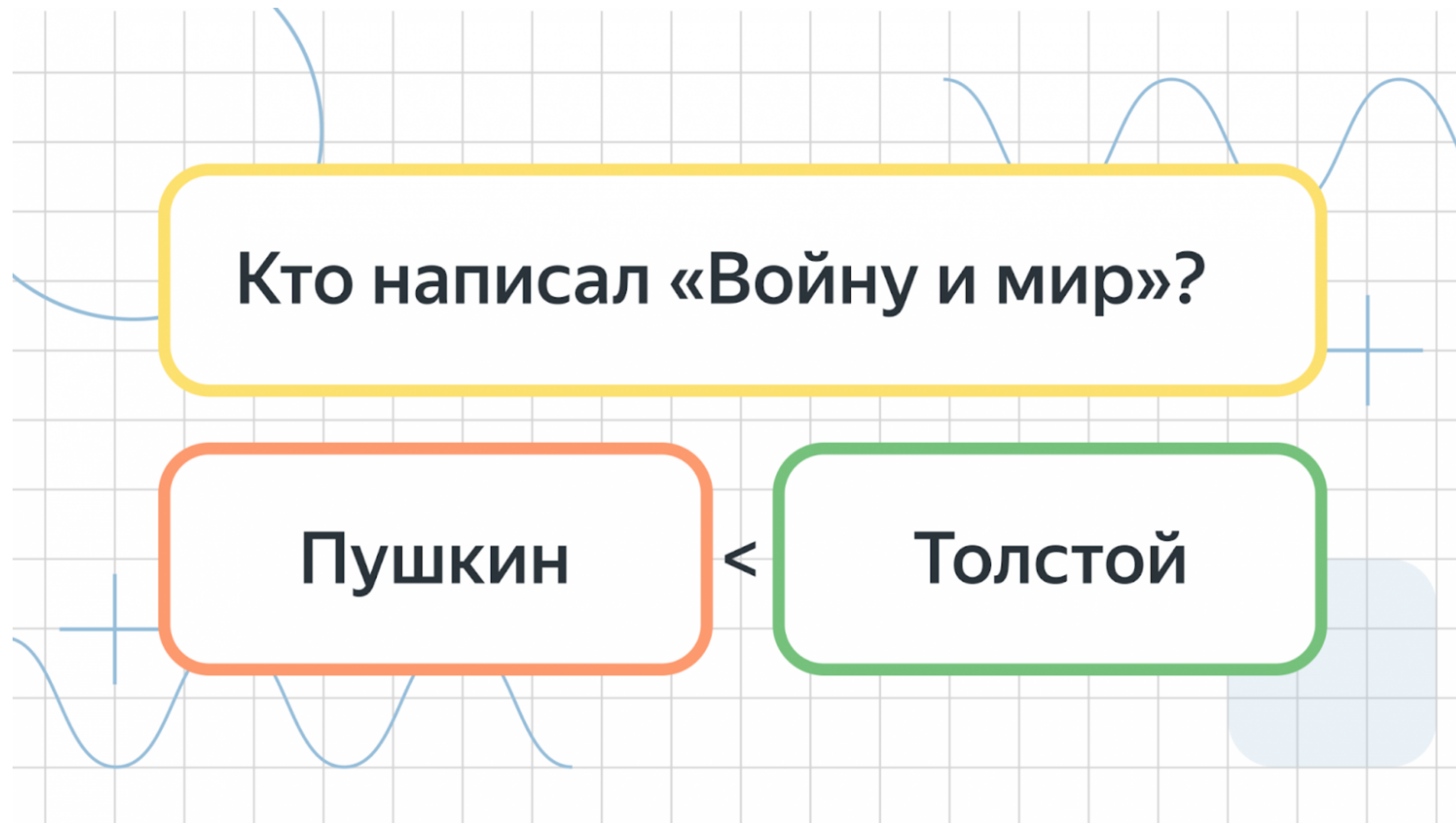


Продуктовый срез

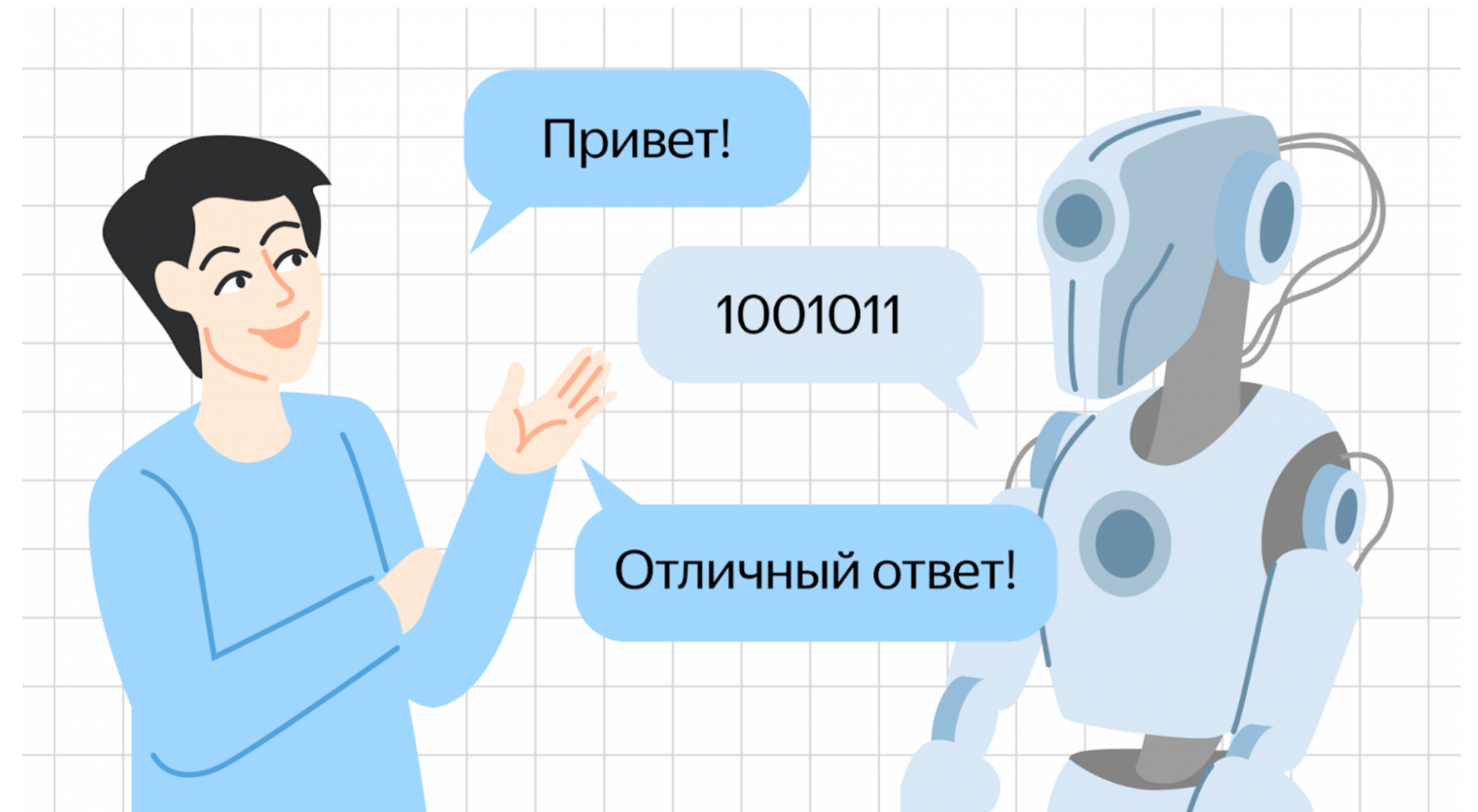
Reword

Как оценивать качество

Pairwise оценка

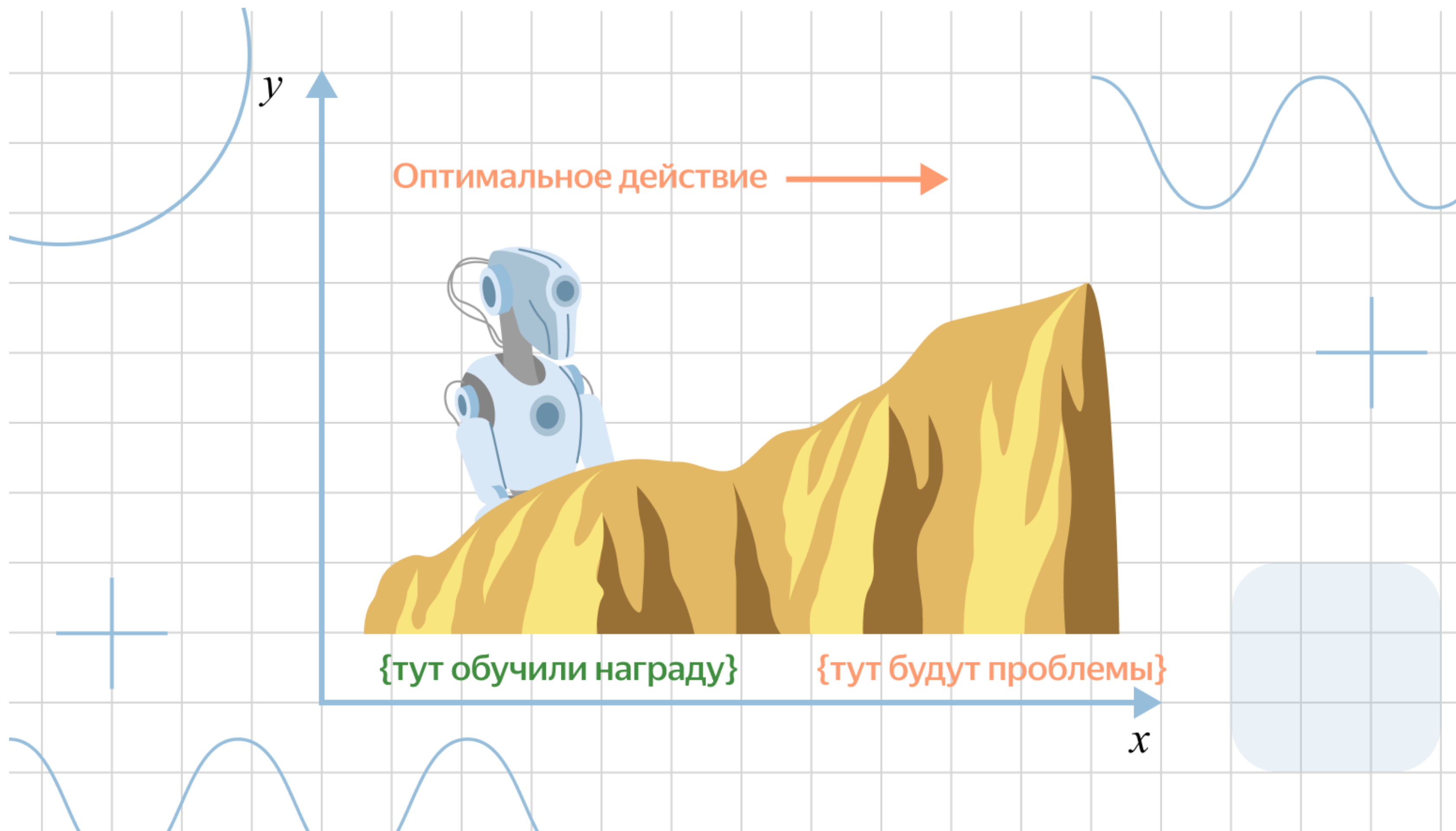


Pointwise оценка



Reward

Доменный сдвиг



Reword

Почему pointwise плох

- Pointwise оценки менее гибкие: новые неожиданные генерации случайно попадают в какой-то класс: sbs - надо переучить, pointwise - надо править инструкцию
- Когда в pointwise много аспектов тяжело контролировать качество
- Тем не менее в каких-то понятных задачах pointwise вполне себе работает, примеры: подтвержденность/фактологичность/орфография

Bradley-Terry

Вводим

- R - реворд модель оценивающая “силу” ответа
- x - инстракт
- y1 - правый ответ
- y2 - левый ответ

$$P(y_1 > y_2 | x) = \frac{\exp(R(x, y_1))}{\exp(R(x, y_1)) + \exp(R(x, y_2))}$$

$$L(x, y_1, y_2) = - (I(y_1 > y_2) \cdot \log(p(y_1 > y_2 | x)) + I(y_2 > y_1) \cdot \log(p(y_2 > y_1 | x)))$$

Bradley-Terry

Проблемы

- Собрать разметку ранжирующую пары хорошего качества очень сложно, 70% точности считается уже хорошо
- RL учится на sparse сигнал, оверфит наступает достаточно быстро
- Модель Бредли-Тэрри не разрешает циклы. Невозможно чтобы было $A > B$, $B > C$ и при этом $C > A$, при этом существует Парадокс Кондорсе

Метрики и реворды

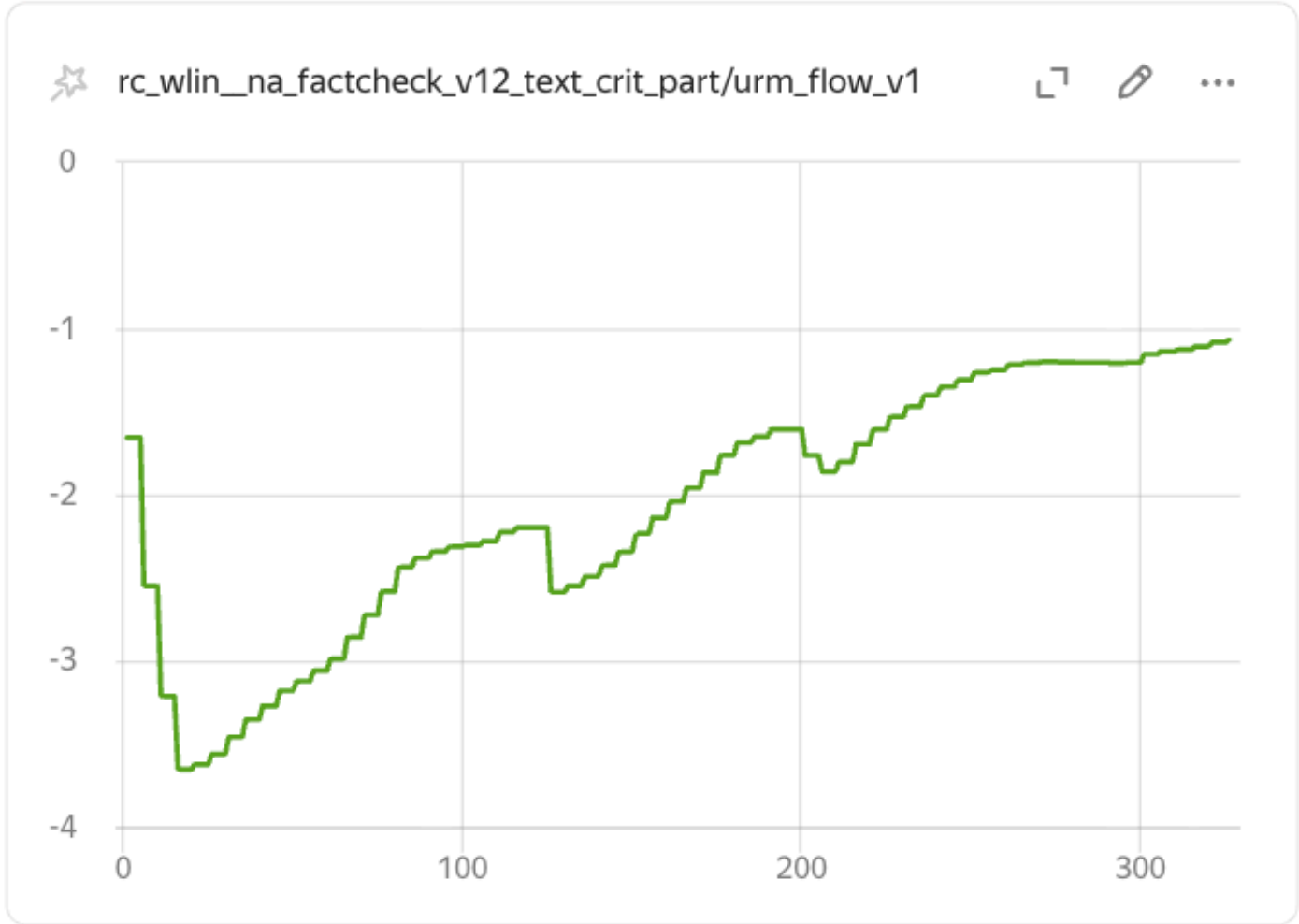
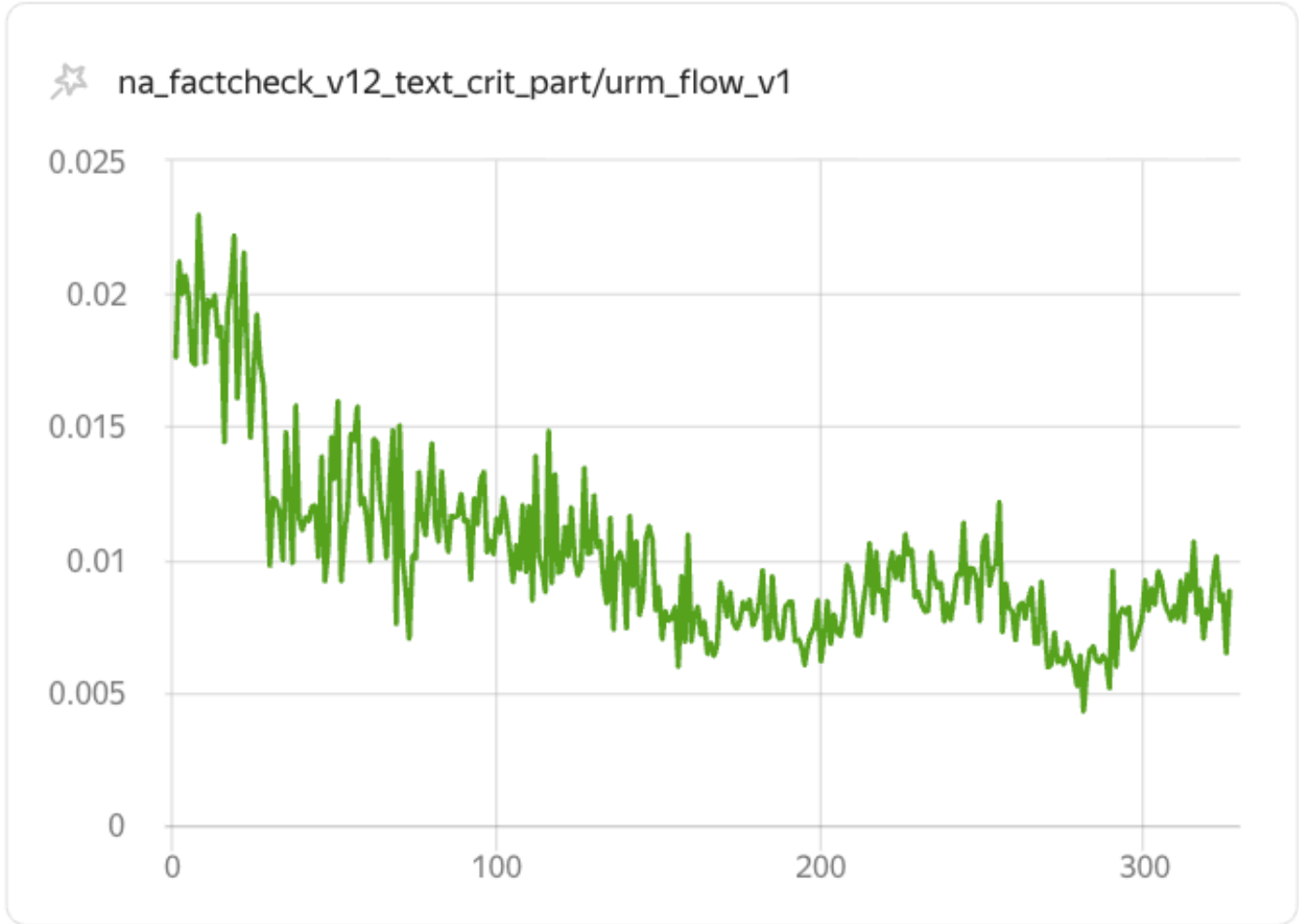
- Полезность - ответ понятен и решает задачу пользователя
- Фактчек - в ответе нет вранья
- Тон - тон ответа, его приятность
- Обогащения - картинки/видео/еком/гео вставлены в ответ по месту
- Этика
- Красота
- Память - память о пользователе использована уместно
- ...

Срезы

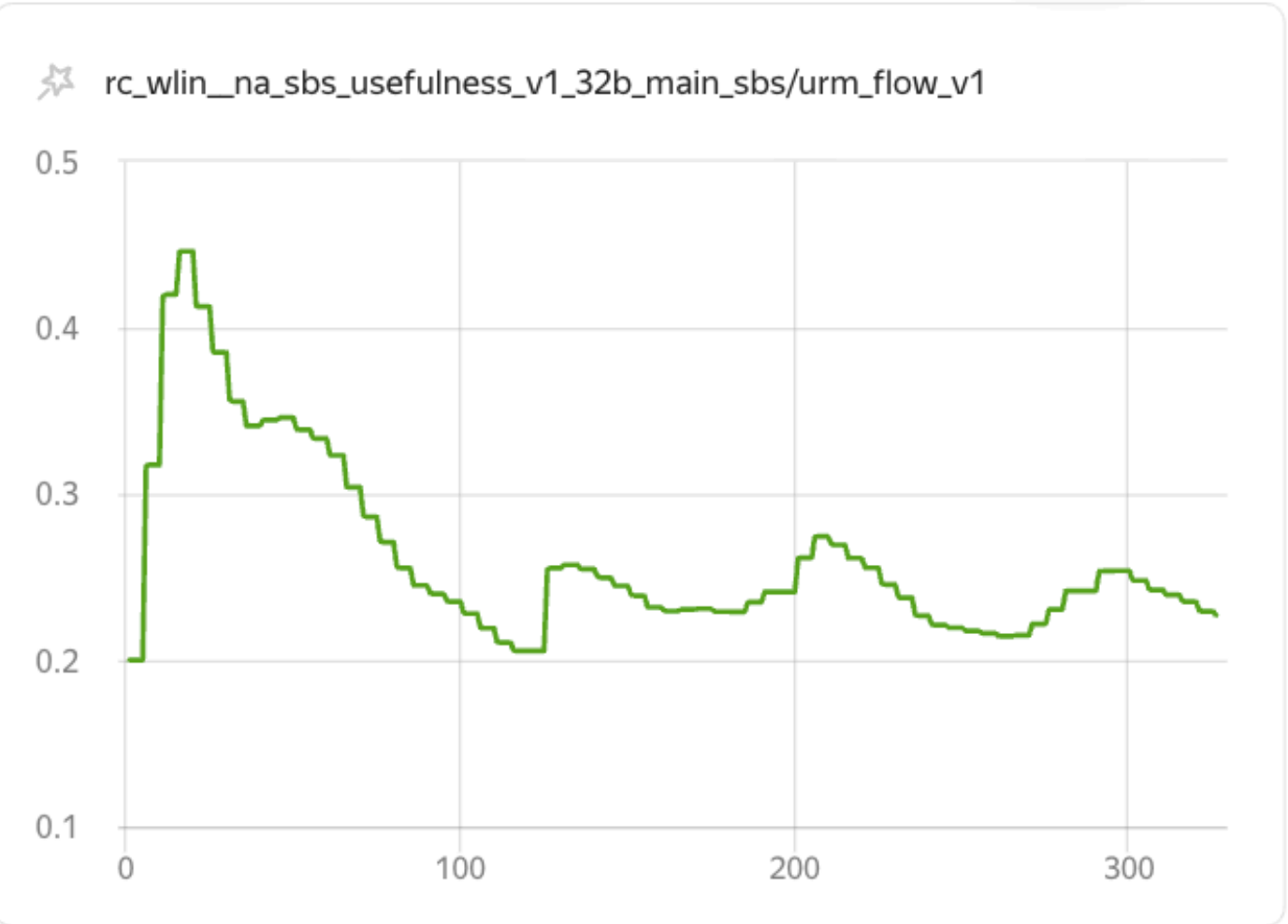
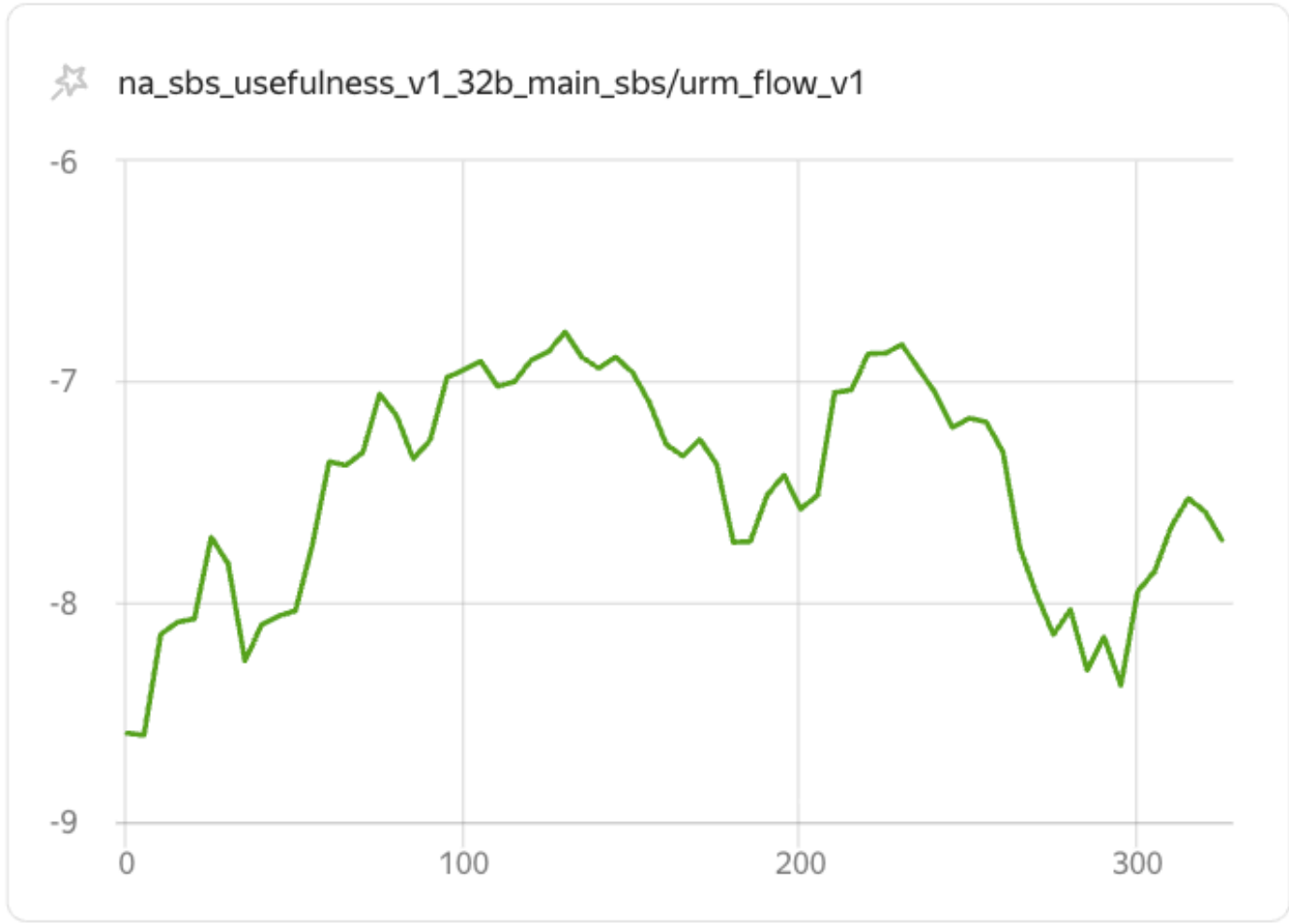
- Интеграл - поток запросов в Алису, хотим иметь минимум плохих ответов за минимальную стоимость
- Еком - хотим помогать человеку понимать что купить и искать товары, GMV
- Эмо - срез где нужно проявлять эмоции/сочувствие/итд
- Этика - нельзя делать плохо :)
- VLM Hard - сложный запрос со сложной картинкой
-
- Надо оптимизировать это все в одном обучении - для RL надо одно число

Автоформула

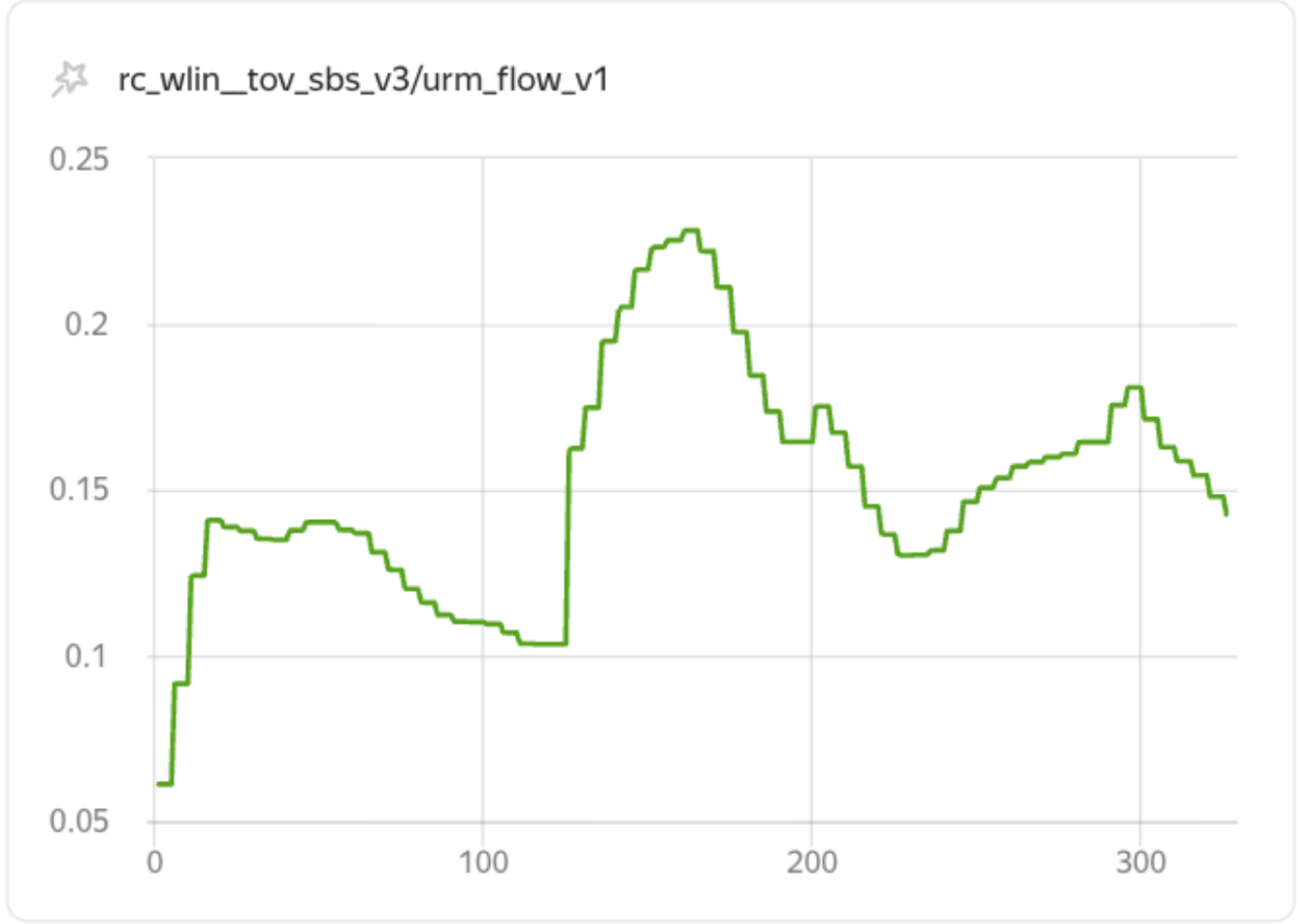
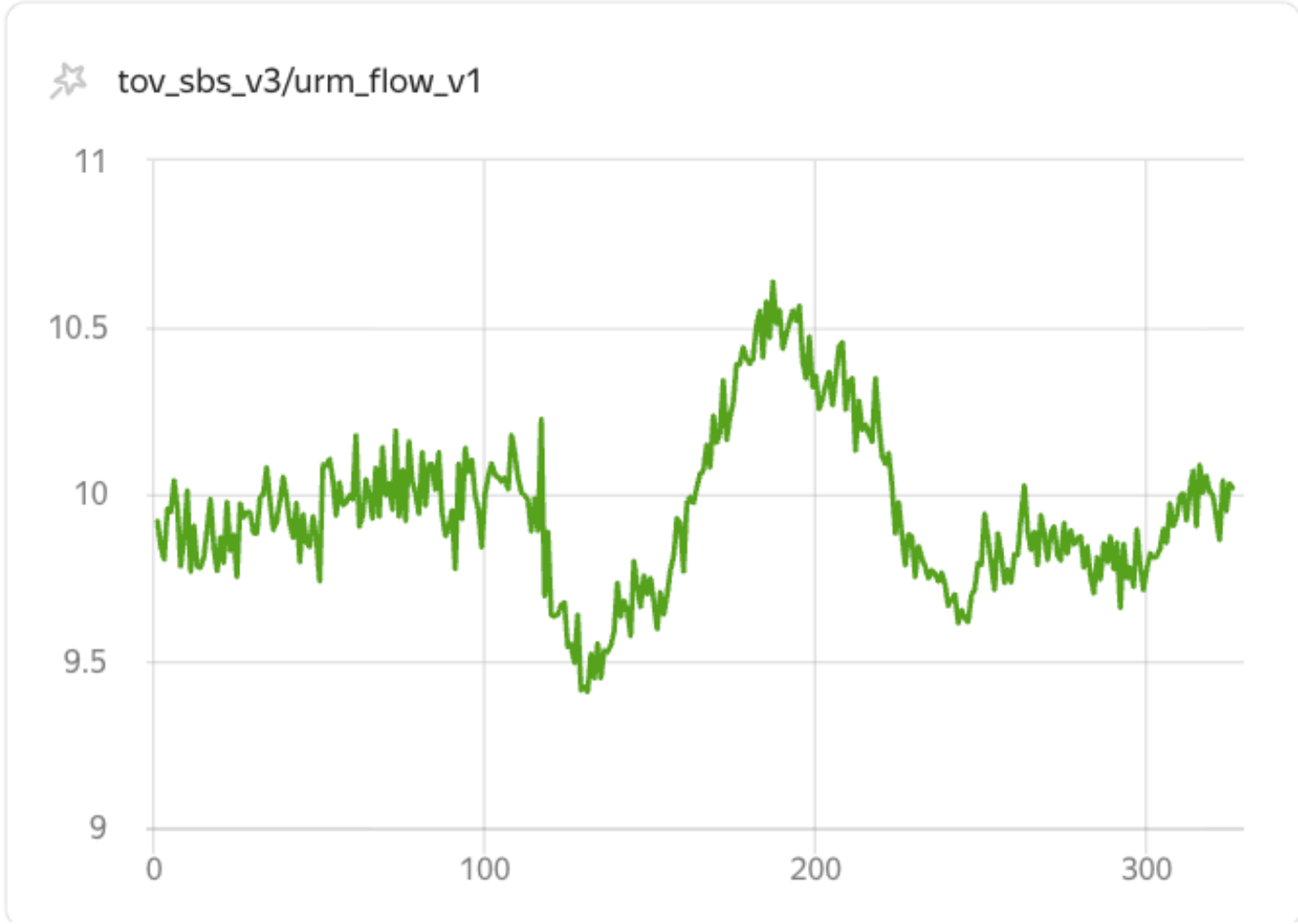
Оптимизирует фактчек



Полезность чуть улучшить



TOV не трогать, держать на месте



Рубрики — reward для открытых задач

RLVR generalization: prompt-specific criteria → judge scores → scalar reward.

Что такое rubric reward

Prompt-specific набор положительных и отрицательных критериев. Judge проверяет каждый criterion; оценки агрегируются в один reward.

$$R = \sum_i w_i \cdot \text{score}(c_i)$$

RLVR — частный случай: один criterion = exact match / tests.

Пример: запрос → rubric → reward

«Составь ужин для четырёх человек за 45 минут и до 3 000 ₽. Один гость — вегетарианец, у другого аллергия на орехи.»

+3	БЕЗОПАСНОСТЬ	нет орехов; учтён риск cross-contact
+2	РАЦИОН	есть полноценное vegetarian блюдо
+2	БЮДЖЕТ	список продуктов ≤ 3 000 ₽
+2	ВРЕМЯ	реалистичные шаги укладываются в 45 минут
−3	PITFALL	аллерген или ограничения проигнорированы

reward = Normalize(Σ passed weights)

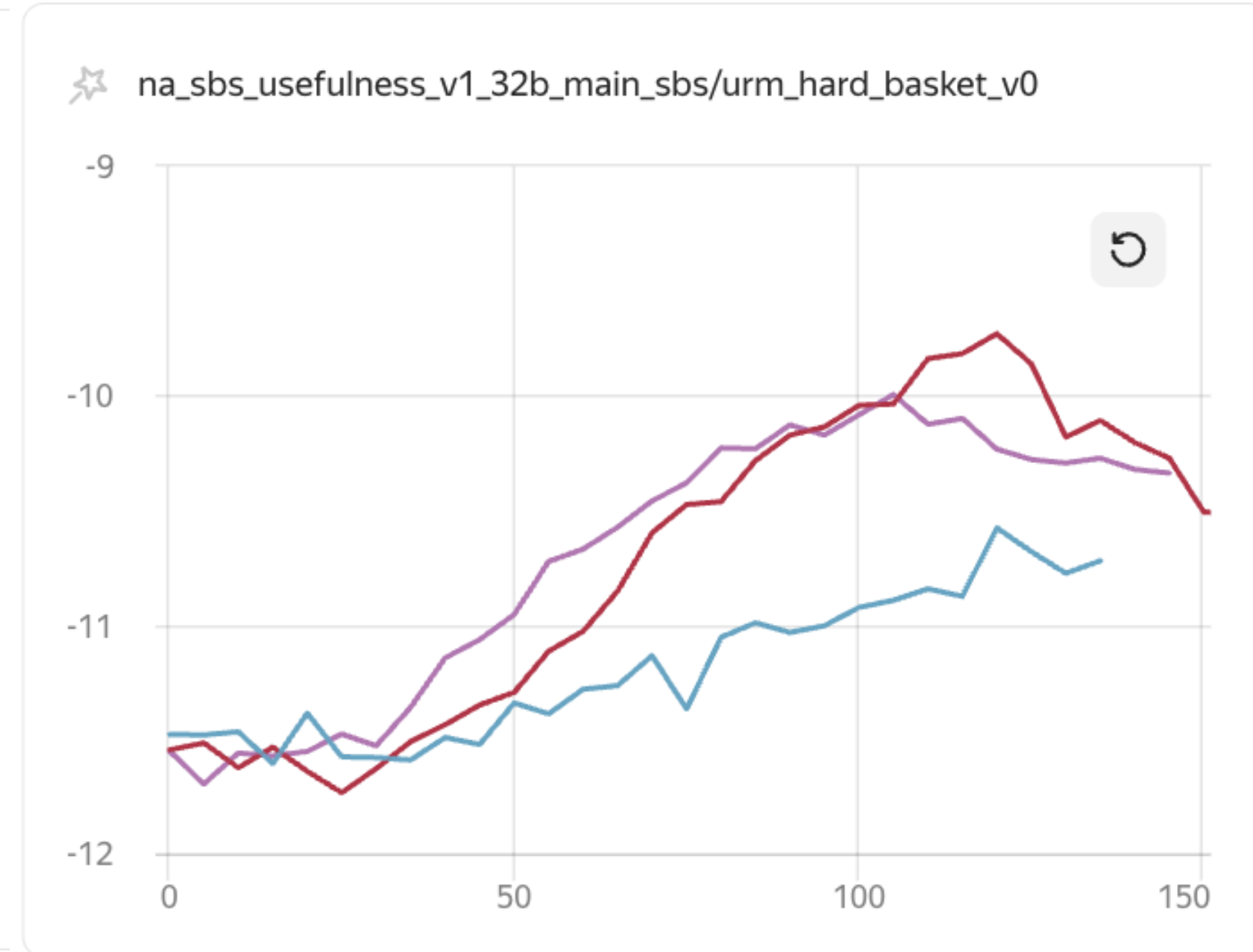
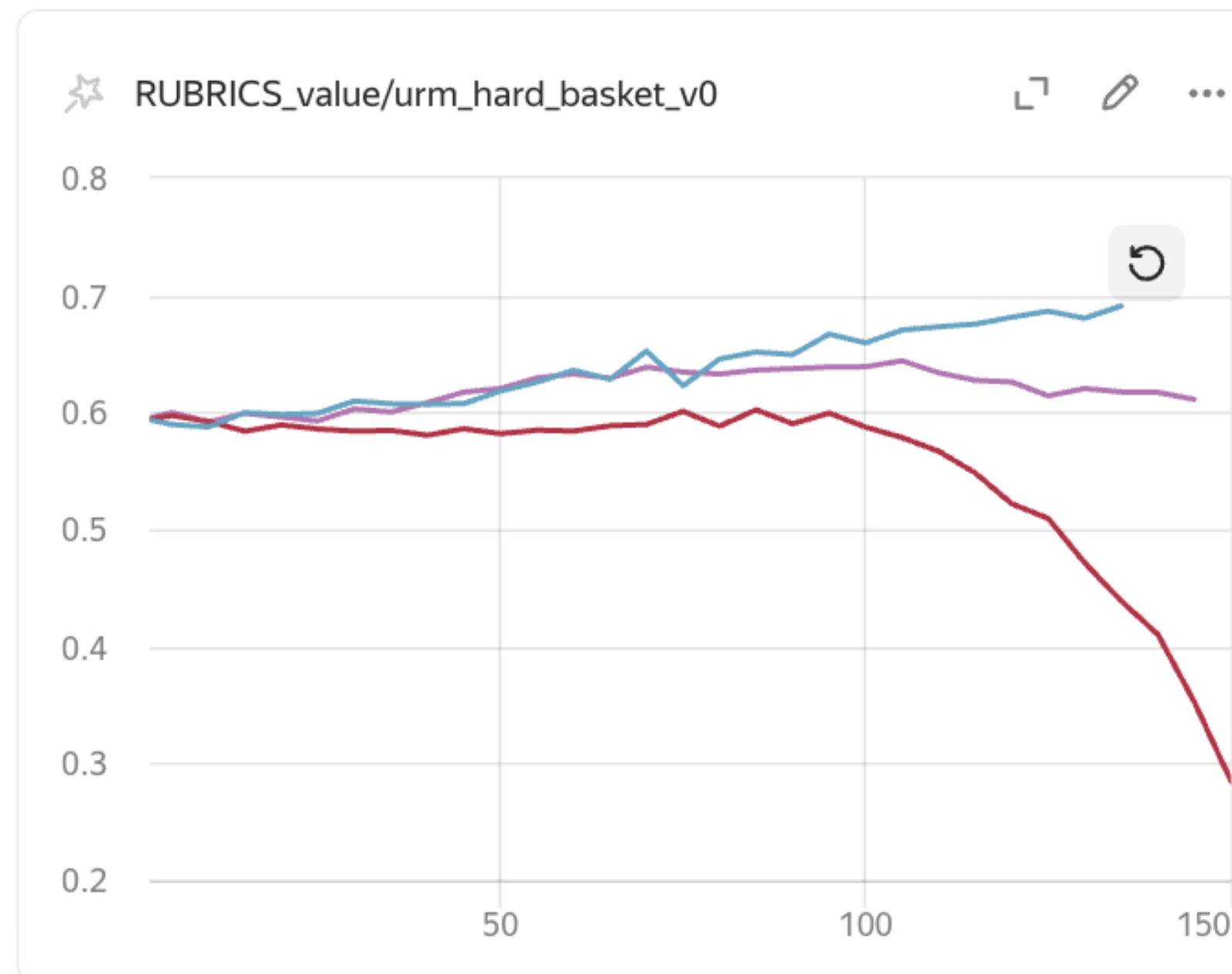
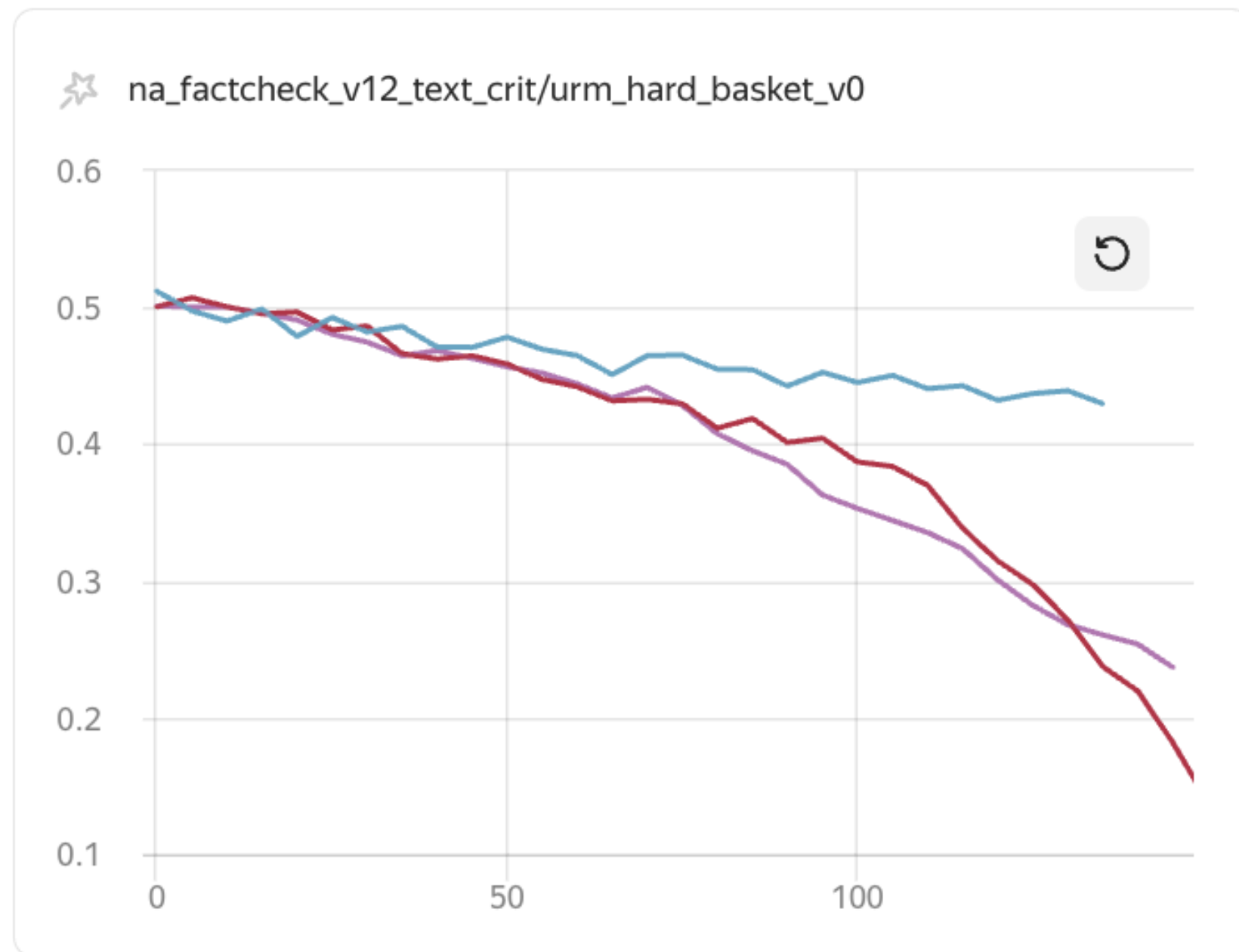
ОСНОВНЫЕ ПЛЮСЫ

- Открытые задачи получают task-specific partial credit.
- Reward интерпретируем: видно, какой criterion сработал.
- Hidden / per-instance rubrics убирают одну статичную цель — оверфит сложнее.
+ верификация рубрик с точностью >90% не дает легко оверфитнуться

ОСНОВНЫЕ МИНУСЫ

- Это всё ещё проху: слабый judge или пробелы rubric → reward hacking.
- Checklist gaming: verbosity вместо качества. Оптимизирует полноту
- Могут со временем терять актуальность
- Как понять что рубрики стали лучше ?

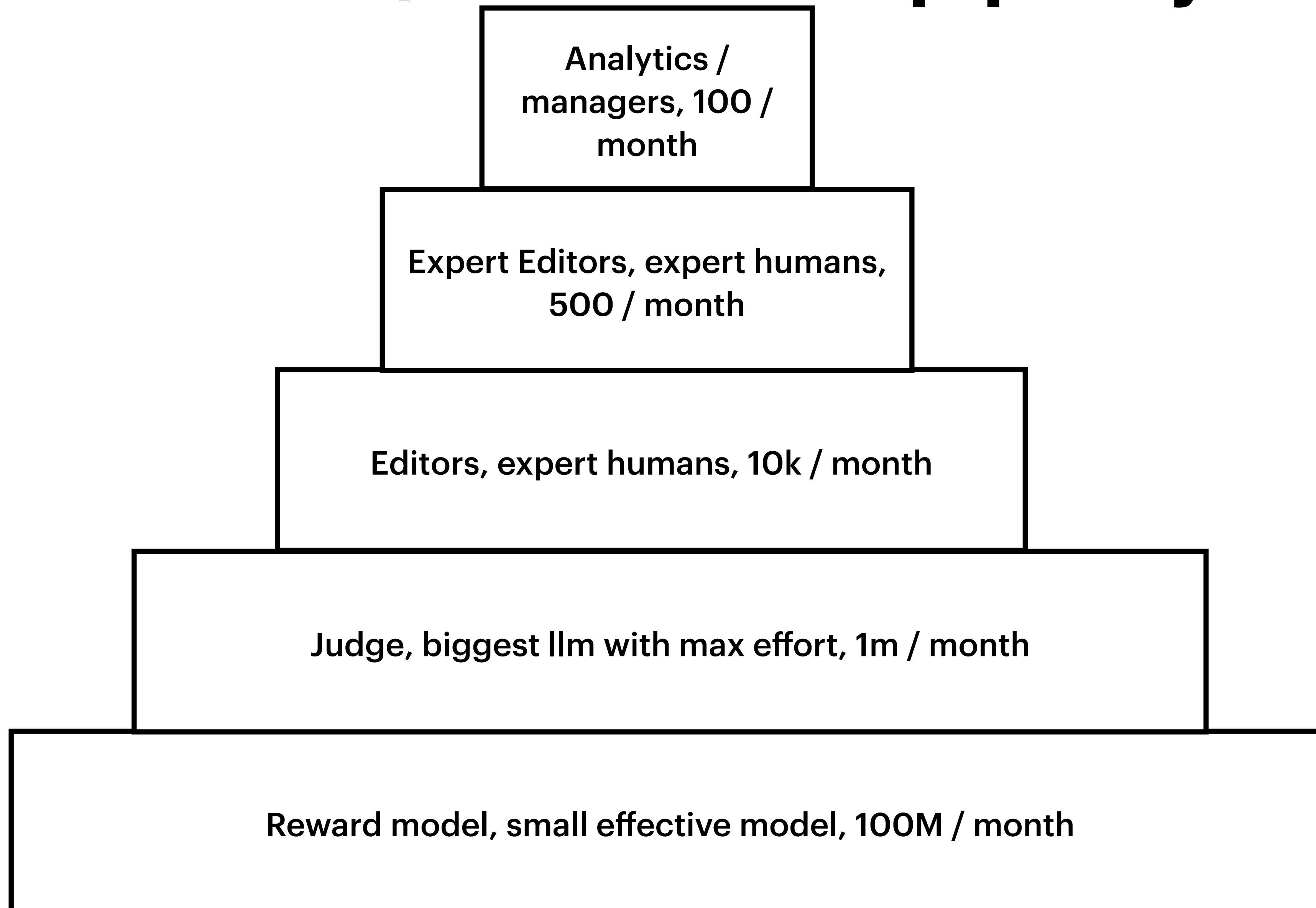
Правда ли нужны рубрики



- Голубой - рубрики + фактчек
- Красная - сбс + фактчек
- Фиолетовая - рубрики + сбс + фактчек

Оверфит

Что вообще можно оверфитнуть ?



Пример с кодом

- Editors - люди проверяющие код с таймаутом
- Golden target - тесты
- Reward - модель учившаяся на разметку людей

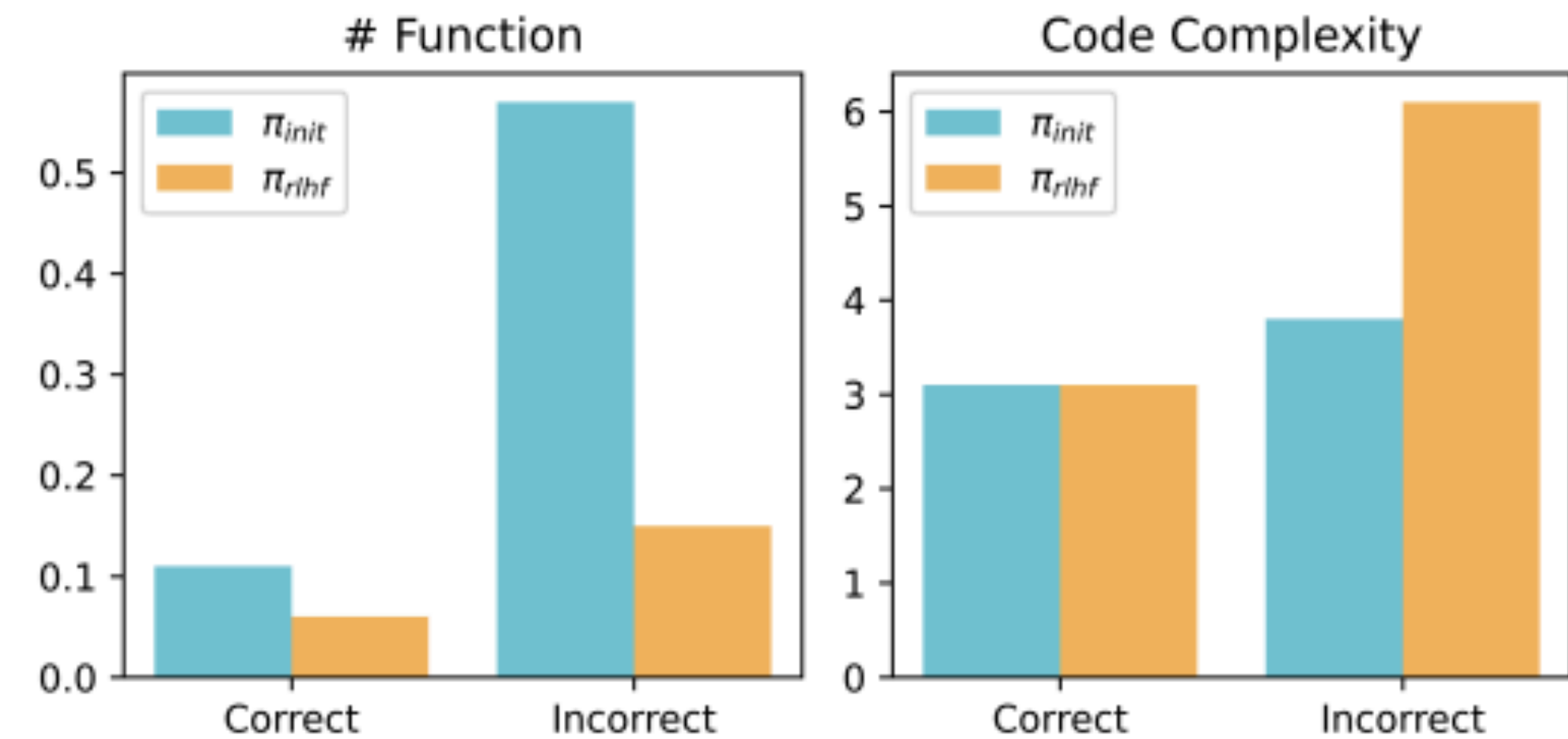


Figure 7: π_{rlhf} tends to generate less readable incorrect programs that contain fewer helper functions and have higher code complexity.

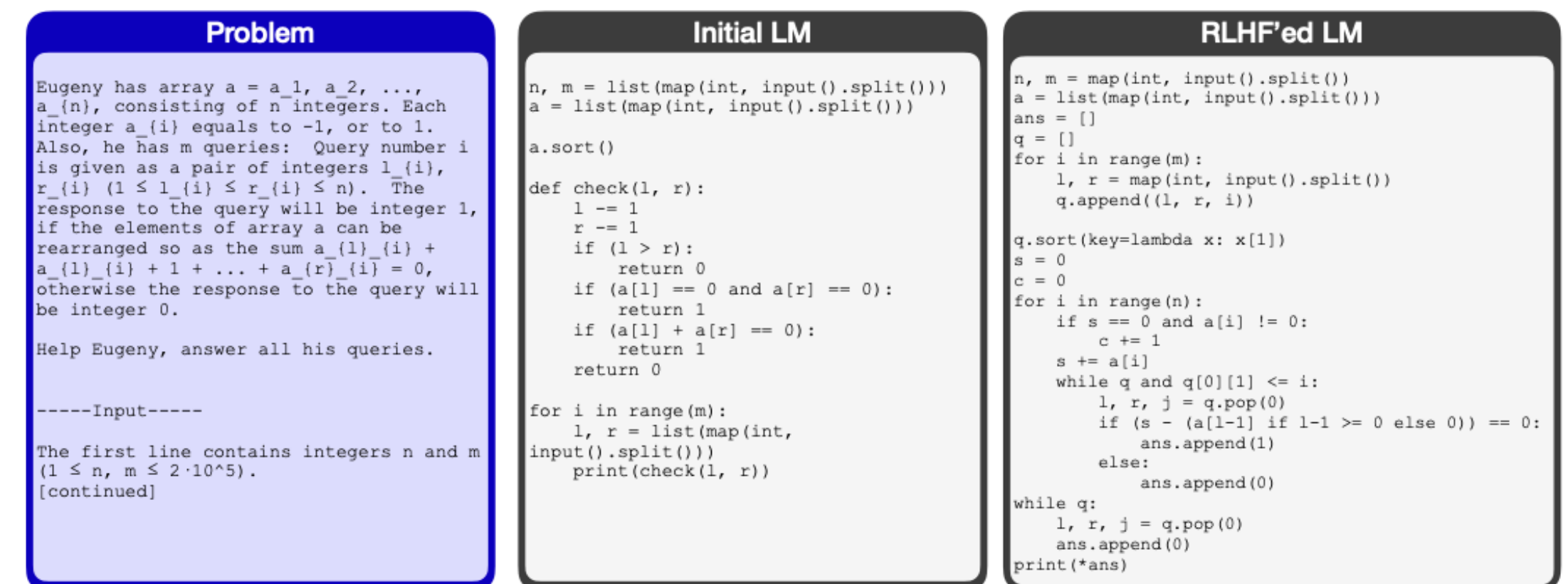


Figure 6: π_{rlhf} mislead humans by generating less readable programs that still pass all evaluator-written unit tests. In contrast, π_{init} generates more modular programs and often fails on the first evaluator-written unit test. This happens even when π_{rlhf} obtains a substantially lower unit test passing rate on the original APPS dataset than π_{init} (26.8% vs. 58.3%).

Паттерны

Хороший вопрос. Я подобрала пару ноутбуков — подсказать, какой взять под бюджет?

Тут видны проблемы модели?

Паттерны

Хороший вопрос. Я подобрала пару ноутбуков — подсказать, какой взять под бюджет?

Хороший вопрос! Я нашла маршруты по городу — хотите, подскажу, что посмотреть за день?

Хороший вопрос. Средняя температура на поверхности Марса — около минус 60 градусов.

А тут ?

Паттерны

batch



group 1



group 2



group 3



group 4

d_0 ...

r_0

d_1

r_1

d_2

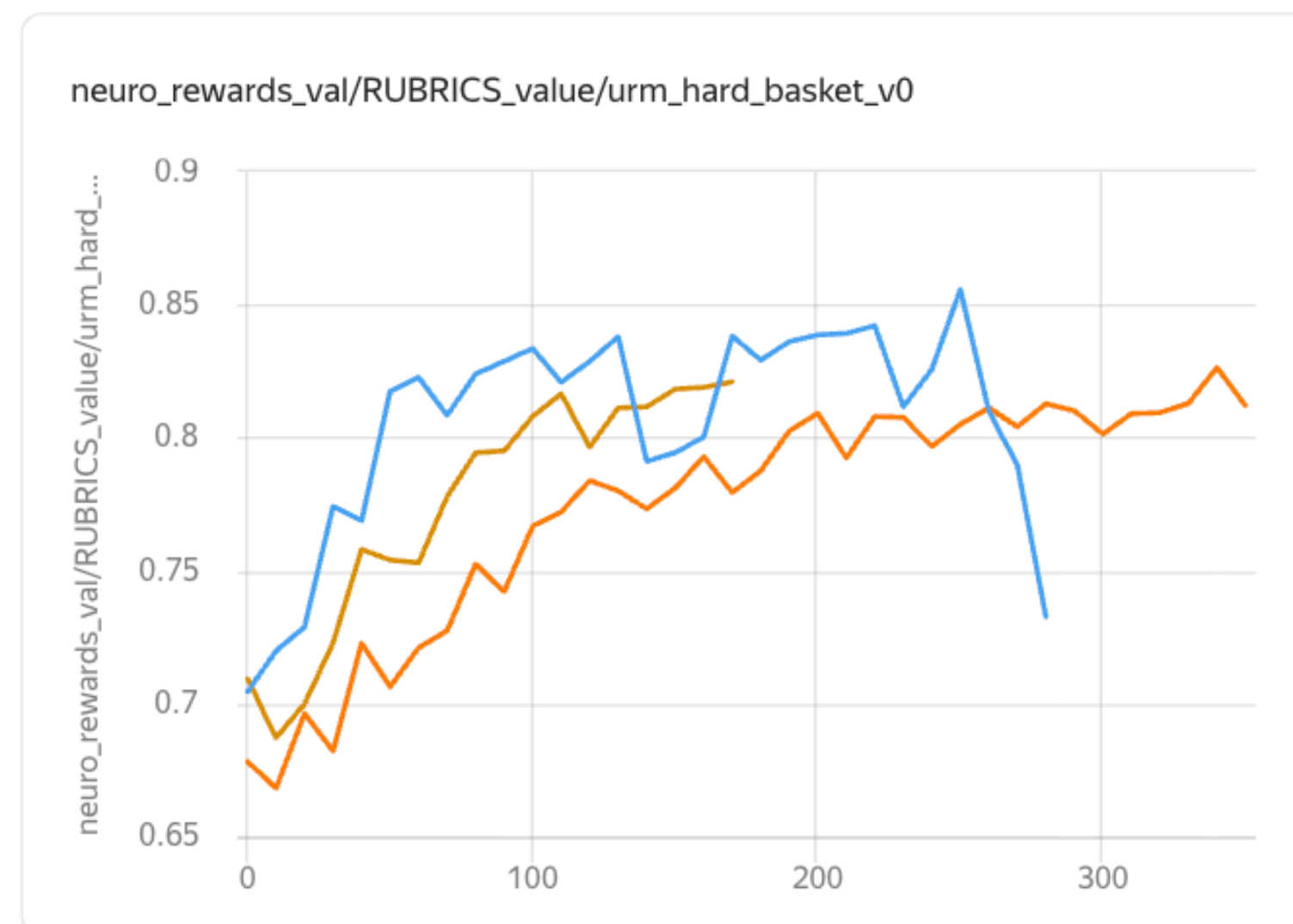
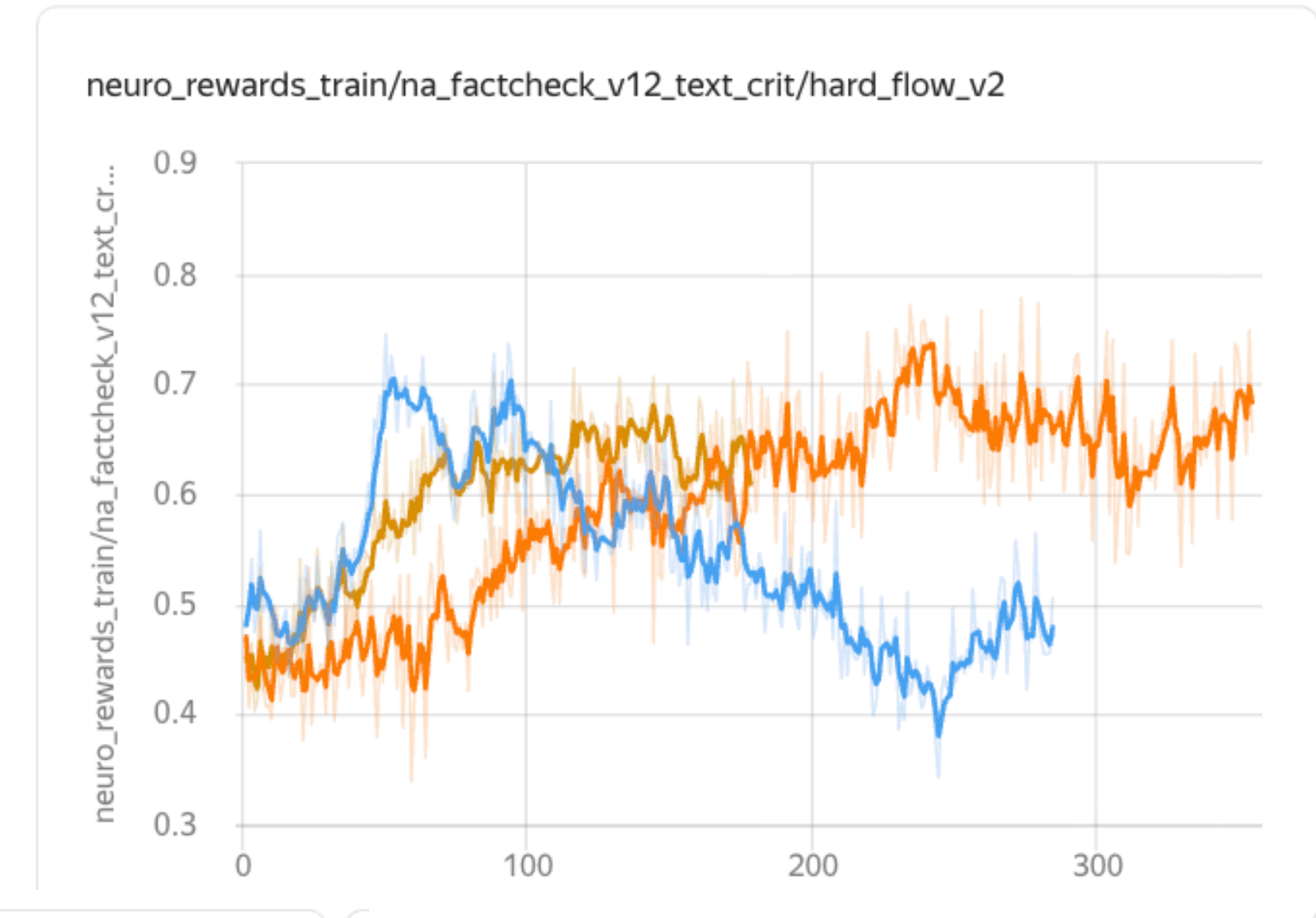
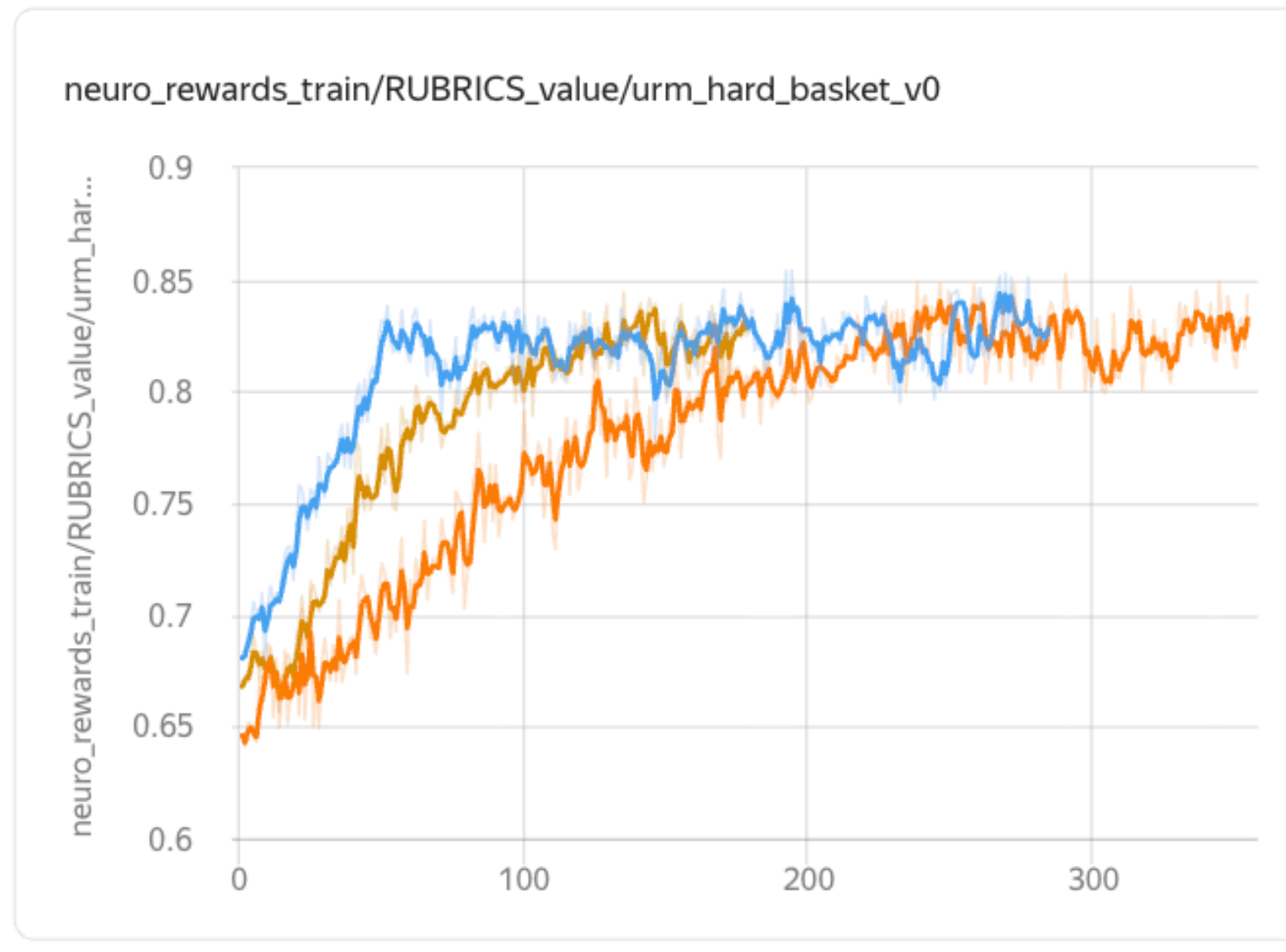
r_2

d_3

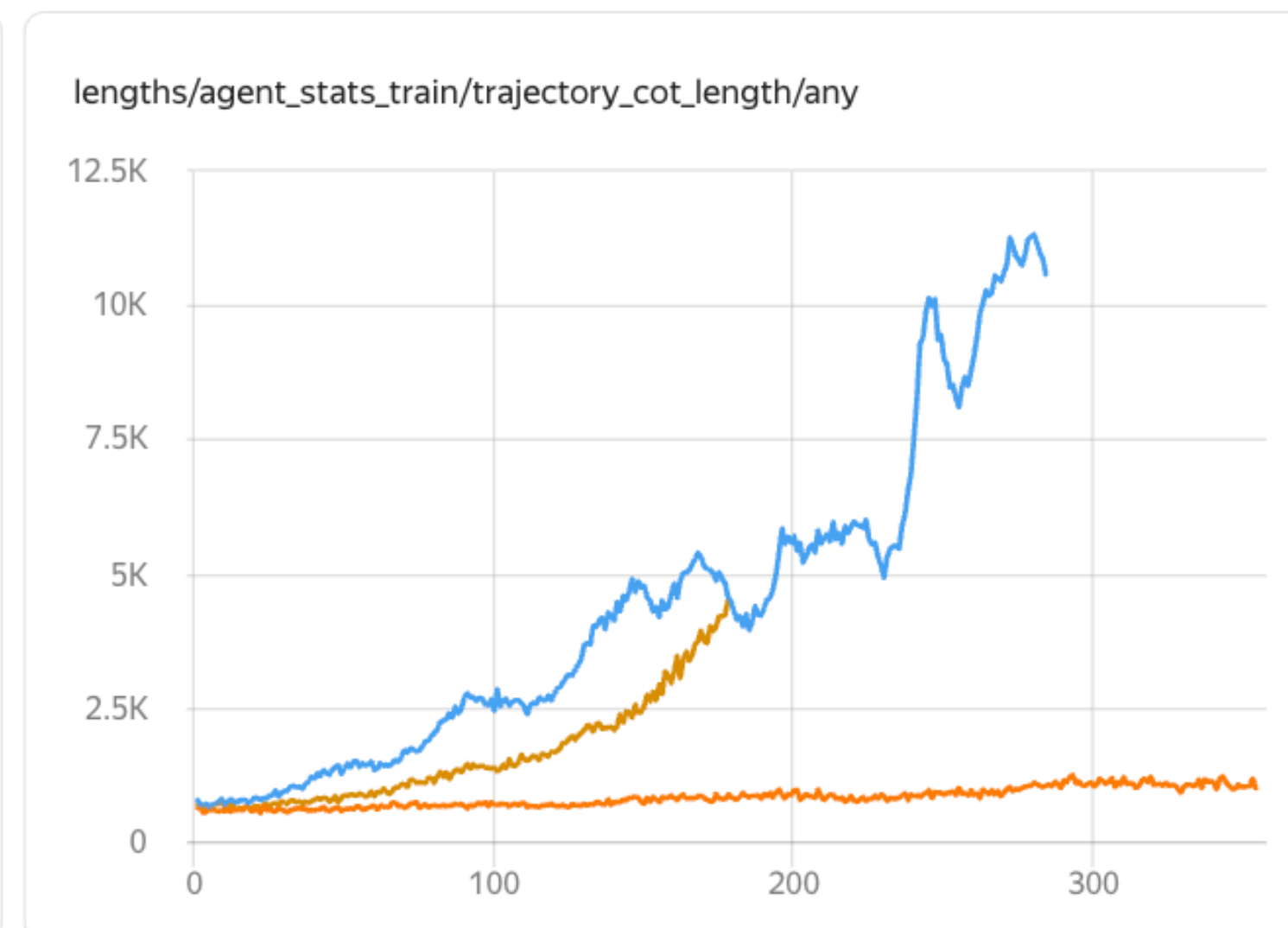
r_3

$$new_r_i = r_i - penalty \cdot d_i * std(r_0, r_1, r_2, r_3)$$

Бывает ли оверфит ?



Рубрики, train



Фактчек, процент вранья

Рубрики, валидация

Длина ризонинга

Проблемы в базовых способностях

Проблемы с временем

Как думаете с какой точностью наша лучшая модель это решает ?

Расставь эти варианты по порядку — от самого раннего момента к самому позднему.

T003) 2026-06-21 06:28 UTC
T011) 2026-06-21 07:37 UTC
T012) 2026-06-21 08:11 UTC+0
T017) 2026-06-21 08:53 UTC
T007) 21 июня 2026, 14:08 в Новосибирске
T008) 2026-06-21 02:23 Нью-Йорк
T016) 2026-06-21 09:50 UTC
T002) 2026-06-21 16:43 Новосибирск
T014) 21 июня 2026, 15:43 в Аделаиде
T006) 2026-06-21 11:56 Дубай
T013) 21 июня 2026, 11:01 в Лондоне
T004) 2026-06-21 18:04 Окленд, Новая Зеландия
T009) 2026-06-21 08:02 UTC
T001) 2026-06-21 12:00 UTC+3
T005) 21 июня 2026, 09:20 в Лондоне
T010) 20 июня 2026, 23:57 в Лос-Анджелесе
T015) 2026-06-21 14:17 Новосибирск

Смотрим разбивку по срезам

#	dssm_cluster ⓘ ▲	▼ total ⓘ	▲ Kimi K3 ⓘ	Our model	▲ ties ⓘ
1	"education"	38	20	14	4
2	"how_to"	27	13	11	3
3	"medicine"	23	11	10	2
4	"law"	20	7	11	2
5	"none_of_the_above"	17	9	6	2
6	"finance"	15	11	3	1
7	"info_ru"	14	7	6	1
8	"ecom"	11	4	6	1
9	"cs"	10	6	3	1
10	"games"	7	6	1	0
11	"music"	3	2	1	0
12	"auto"	3	1	1	1
13	"travel"	3	2	1	0
14	"news"	2	2	0	0
15	"real_estate"	2	1	1	0
16	"services"	1	0	1	0
17	"movies_and_series_tv_online"	1	0	1	0
18	"sports"	1	1	0	0

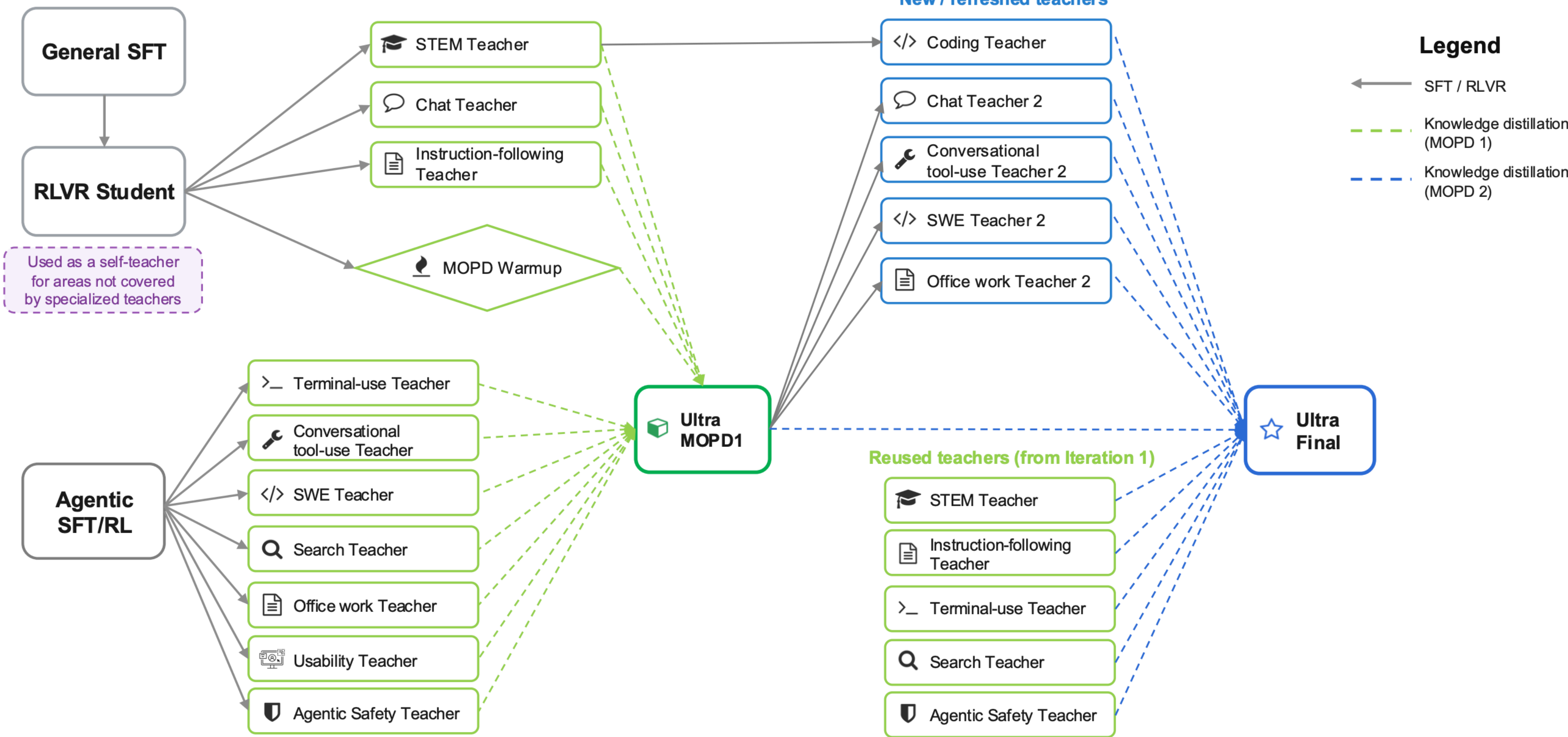
Конец ?

MOPD

Prep: Build RLVR Student

Iteration 1 (MOPD 1)

Iteration 2 (MOPD 2)



MOPD

$$\mathcal{J}_{\text{MOPD}}(\theta) = \sum_{i=1}^N \lambda_i \mathbb{E}_{q \sim \mathcal{D}_i, y \sim \pi_{\theta}(\cdot | q)} \left[\sum_{t=1}^H \left(\log \pi^{T_i}(y_t | s_t) - \log \pi_{\theta}(y_t | s_t) \right) \right]$$

π_{θ} — обучаемая модель-студент,

π^{T_i} — учитель, специализированный на домене i ,

$\mathcal{D}_i$ — распределение запросов домена i ,

λ_i — вес домена i в обучении,

$y \sim \pi_{\theta}(\cdot | q)$ — ответ, сгенерированный самим студентом.

Спасибо за внимание